Hochschule Aalen

Bachelorarbeit
Studiengang : Medieninformatik

JSONPath-Abfrage über YottaDB

von

Daniel Föhl

83522

Betreuender Professor: Prof. Dr. Winfried Bantel
Zweitprüfer : Prof. Dr. Christian Heinlein

Abgabedatum : 31. August 2025

Eidesstattliche Erklärung

Hiermit erkläre ich, **Daniel Föhl**, dass ich die vorliegenden Angaben in dieser Arbeit wahrheitsgetreu und selbständig verfasst habe.

Weiterhin versichere ich, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben, dass alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und, dass die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

Ort, Datum Unterschrift (Student)

Kurzfassung

Diese Bachelorarbeit wird im Rahmen des Studiengangs Medieninformatik durchgeführt. Das Projekt `JSONPath-Abfrage über YottaDB` entwickelt ein Command Line Interface (CLI)-Werkzeug zur lesenden Abfrage von Daten aus einer YottaDB (YDB). Das Tool nimmt JSONPath-Ausdrücke entgegen, parst eine Teilmenge der Sprache mit Flex/Bison zu einem Abstract Syntax Tree (AST) und übersetzt diesen AST in eine YottaDB-Referenz (Globalname + Subskripte). Anschließend wird die entsprechende YDB-Abfrage ausgeführt und das Ergebnis im CLI zurückgegeben.

Die Hauptaufgaben dieser Arbeit umfassen:

- Die Definition der Anforderungen und Auswahl eines Teilumfangs von JSONPath
- Den Entwurf von Grammatik-Ausschnitten und AST-Knotentypen
- Die Implementierung einer robusten Pipeline
  (Scanner → Parser → AST → AST-zu-YDB-Translator → CLI-Ausgabe)
- Tests zur Korrektheit und Gebrauchstauglichkeit

Die Grundimplementierung legt den Schwerpunkt auf eine robuste und nachvollziehbare Kernpipeline. Anstelle einer vollständigen Sprachabdeckung steht zunächst die zuverlässige Ausführung der zentralen Konstrukte im Vordergrund. Damit entsteht eine stabile Basis, auf der weiterführende Funktionen schrittweise und kontrolliert ergänzt werden können.

Im Anschluss werden mögliche Erweiterungen skizziert, etwa eine sukzessive Ausweitung der JSONPath-Unterstützung, die Integration komplexerer Abfrageformen (Wildcards, Filter, Slices) sowie die längerfristige Annäherung an eine vollständige RFC-Konformität.

Inhaltsverzeichnis

Eidesstattliche Erklärung i

Kurzfassung ii

Inhaltsverzeichnis iii

Abbildungsverzeichnis vii

Tabellenverzeichnis viii

Abkürzungsverzeichnis x

Glossar xi

1 Einleitung 1

1.1 Motivation 1

1.2 Problemstellung und -abgrenzung 2

1.3 Ziel der Arbeit 3

1.4 Vorgehen 4

2 Grundlagen 5

2.1 JSON – Format für strukturierte Daten 5

2.2 JSONPath – Pfadbasierte Abfragesprache 6

2.2.1 Standardisierung und Varianten 6

2.2.2 Struktureller Aufbau von JSONPath-Ausdrücken 6

2.2.3 Syntaxelemente von JSONPath 8
2.2.4 Beispiele 9
2.3 YottaDB – Überblick 9
2.3.1 Globals und Subskripte 10
2.3.2 Routinen 11
2.4 Mapping JSONPath → YottaDB 12
2.4.1 Voraussetzungen und Notation 12
2.4.2 Basismapping 12
2.4.3 Typkodierung von Subskripten 13
2.5 Referenz Beispiel 14
2.6 Parser 16
2.6.1 Formale Sprachen und Chomsky-Hierarchie 16
2.6.2 Lexikalische und syntaktische Analyse 17
2.6.3 Abstract Syntax Tree (AST) 18
2.6.4 Besonderheiten des Parsers 18
2.7 Fehlermodell 19

3 Problemanalyse 20

3.1 Ist-Zustand und Lücke in YottaDB 20
3.2 Anforderungen und Randbedingungen 20
3.3 Alternativen und Abwägungen 21

4 Lösungskonzept 22

4.1 Architekturübersicht 22
4.2 Architekturpipeline 23
4.3 Sprachmodell 23
4.4 Übersetzung zu YottaDB 24

4.5 Zugriffsschicht 25
4.6 Fehlermanagement 25
4.7 CLI-Design 26
4.8 Erweiterbarkeit 27

5 Implementierung 28

5.1 Code-Struktur 28
5.2 Scanner 29
5.3 Parser 31
5.4 AST-Implementierung und Pretty-Printer 35
5.5 YDB-Translator 38
5.5.1 Erstellen der YDB-Referenz 38
5.5.2 Traversierung 39
5.5.3 Hilfsfunktionen 41
5.6 Build-Prozess 43
5.7 Zugriffsschicht und CLI 44

6 Inbetriebnahme 47

6.1 Voraussetzungen und Installation 47
6.2 Beispieldaten und Referenzaufrufe 49
6.2.1 Überblick über das Beispieldataset 49
6.2.2 Abfragen über das CLI 50
6.2.3 Alternative Roots 51
6.2.4 Fehlerfälle und Negativtests 51
6.2.5 Manuelle Tests der Bridge 51
6.2.6 Debugging mit JP_DEBUG 52
6.2.7 Interaktiver Modus 52

6.3 Konfiguration und Reproduzierbarkeit 53

7 Evaluierung **55**

7.1 Methodik und Metriken 55

7.2 Korrektheit und Fehlertoleranz 56

7.3 Gebrauchstauglichkeit 58

7.4 Ergebnisse und Diskussion 60

8 Ausblick **64**

8.1 Funktionale Erweiterungen 64

8.2 Standardkonformität und Performance 65

8.3 Integration und Verbreitung 65

8.4 Darüber hinaus 66

Literatur **67**

Abbildungsverzeichnis

2.1 Struktureller Aufbau eines JSONPath-Ausdrucks 7
2.2 Verschachtelte Darstellung der Chomsky-Hierarchie 16
2.3 Beispiel AST für einen JSONPath-Ausdruck 18

4.1 Pipeline vom JSONPath-String bis zur YottaDB-Ausgabe 23
4.2 Übersetzung eines JSONPath-Ausdrucks auf eine YottaDB-Referenz . 24

5.1 Beispiel abgeleiteter AST 34
5.2 Ablauf von der Referenzbildung bis zur Marker-Auswertung 46

Tabellenverzeichnis

2.1 JSONPath-Syntaxelemente 8

2.2 Beispiele für JSONPath-Ausdrücke im implementierten Teilumfang . 9

2.3 Beispiele: JSONPath → YDB-Referenz → erwarteter Wert 15

4.1 Architekturkomponenten und ihre konzeptionellen Aufgaben 22

5.1 Dateien und Hauptaufgaben der Implementierung 28

5.2 Vom Scanner genutzte Tokens im aktuellen Teilumfang 30

5.3 Unterstützte JSONPath-Konstrukte 32

5.4 Aktuell nicht unterstützte JSONPath-Konstrukte und Fehlermeldungen 33

5.5 AST-Knotentypen im Projekt 36

6.1 Beispieldaten: JSONPath, YottaDB-Referenz und erwarteter Wert . . 49

7.1 Korrektheit für repräsentative Pfade 56

7.2 Negativfälle: tatsächliche Fehlermeldungen (erste Zeile). 57

7.3 End-to-End-Latenz (10 Mittelwerte à 100 Aufrufe). 60

7.4 $DATA-Fälle mit vorhandenem Datensatz; `d=11` ist YottaDB-spezifisch und in reinem JSON nicht darstellbar. 62

Listings

2.1 Setzen und Lesen eines Werts in YottaDB 10
2.2 Beispiel eines JSON-Dokuments 14
2.3 Schematische Ablage in YottaDB 14
2.4 Beispiel einer vereinfachten EBNF-Grammatik für JSONPath 17

5.1 Startregel für JSONPath im Parser 32
5.2 EBNF-Auszug JSONPath-Grammatik 33
5.3 EBNF-Auszug JSONPath-Grammatik 34
5.4 struct ASTNode 35
5.5 Knotenerzeugung 36
5.6 struct ASTNode 37
5.7 Vollständige Funktion `build_ydb_ref` 38
5.8 Traversierer `walk_ast` (aus `query_yottadb.c`) 39
5.9 Traversierer `walk_ast` (aus `query_yottadb.c`) 40
5.10 `buf_cat` (aus `query_yottadb.c`) 41
5.11 `m_escape` (aus `query_yottadb.c`) 41
5.12 `finalize_ref` (aus `query_yottadb.c`) 42
5.13 Die Routine `JPBRIDGE.m` 45

7.1 Messskript zur Latenzbestimmung 60
7.2 Nachweis der $DATA-Fälle d=0/1/10 61
7.3 Technischer Nachweis $DATA=11 in YottaDB 61
7.4 Simulation eines $DATA=11-Knotens in JSON 62

Abkürzungsverzeichnis

ACID	Atomicity, Consistency, Isolation, Durability	9
API	Application Programming Interface	9
AST	Abstract Syntax Tree	ii
CI	Continuous Integration	65
CLI	Command Line Interface	ii
EBNF	Extended Backus-Naur Form	16
GCC	GNU Compiler Collection	55
I/O	Input/Output	2
JSON	JavaScript Object Notation	1
NoSQL	Not only SQL	5
OS	Operating System	63
POSIX	Portable Operating System Interface	19
PostgreSQL	Postgres Structured Query Language	21
RFC	Request for Comments	65
YDB	YottaDB	ii
n. u.	nicht unterstützt	19

Glossar

Parser Analysiert eine Zeichen- oder Tokenfolge anhand einer Grammatik.

Bison Parsergenerator, der aus einer formalen Grammatik C-Code zur Syntaxanalyse erzeugt.

Lexer Modul zur lexikalischen Analyse, das Eingabetext in einzelne Tokens aufteilt.

Flex Werkzeug zur Erzeugung von Lexer-Programmen aus regulären Ausdrücken.

Baumstruktur Hierarchische Datenstruktur aus Knoten mit untergeordneten Knoten (Kindern).

Kindknoten (Kinder) Untergeordnete Elemente in einer Baumstruktur, die direkt einem Elternknoten zugeordnet sind.

Teilbaum Ausschnitt einer Baumstruktur.

Syntaxbaum (AST) Baumstruktur, die die syntaktische Struktur eines Ausdrucks gemäß einer Grammatik darstellt und für die Weiterverarbeitung genutzt wird.

Traversierungsroutine Algorithmus zur systematischen Durchquerung einer Baumstruktur.

Pretty Printer Komponente zur strukturierten und formatierten Ausgabe von Datenstrukturen.

YottaDB Transaktionales, schemaloses Datenbanksystem auf Basis von M (MUMPS).

ACID bezeichnet die klassischen Garantien, die ein Datenbanksystem für Transaktionen bereitstellt.

Global Persistente hierarchische Schlüsselstruktur in YottaDB.

Subskript Index oder Schlüssel, mit dem innerhalb eines YottaDB-Globals auf einen Wert oder Teilbaum zugegriffen wird.

Kanonisierung Vereinheitlichung unterschiedlicher, aber äquivalenter Darstellungen zu einer kanonischen Form.

JPBRIDGE M-Routine, die als Brücke zwischen dem CLI und YottaDB dient.

Marker Standardisierte Ausgaben der Bridge, die vom Hostprogramm ausgewertet werden.

$DATA YottaDB-Funktion, die den Status eines Knotens angibt.

$GET YottaDB-Funktion, die den Wert eines Knotens zurückgibt.

CLI-Tool (jp2ydb) Kommandozeilenprogramm, das JSONPath-Ausdrücke einliest, in YottaDB-Referenzen übersetzt und den Wert zurückliefert.

AST-Ansicht Ausgabeoption des CLI, die den Abstrakten Syntaxbaum des geparsten JSONPath-Ausdrucks zeigt.

Referenzansicht Ausgabeoption des CLI, welche die resultierende YottaDB-Referenz darstellt.

Quiet-Modus Ausgabeoption des CLI, die nur den reinen Wert einer Anfrage zurückgibt, ohne Zusatzinformationen.

Interaktivmodus Modus des CLI, in dem mehrere JSONPath-Ausdrücke nacheinander eingegeben werden können.

Docker Container-Laufzeitumgebung, in der YottaDB für dieses Projekt isoliert ausgeführt wird.

Build-System (Makefile) Automatisiertes Skript zur Erzeugung von Executables aus den Quelltexten (Kompilieren, Linken, Aufräumen).

Regressionstest Wiederholbarer Testlauf mit vordefinierten Abfragen (Positiv- und Negativfälle), um Konsistenz und Korrektheit der Implementierung zu prüfen.

Exit-Codes Standardisierte Rückgabewerte des CLI.

CI-Integration Automatisierte Build- und Testausführung zur Sicherstellung der Korrektheit bei Änderungen.

RFC 9535 Formale Spezifikation von JSONPath (Mai 2023), an der sich die Arbeit teilweise orientiert.

JSONPath Pfadbasierte Abfragesprache zur Navigation in JSON-Datenstrukturen.

1 Einleitung

1.1 Motivation

JavaScript Object Notation (JSON) hat sich als De-facto-Standard für die Repräsentation und den Austausch strukturierter Daten etabliert. In modernen Softwaresystemen werden JSON-Dokumente oft ohne festes Schema gespeichert. So lassen sich veränderliche Datenmodelle flexibel abbilden, ohne dass aufwendige Migrationen nötig werden. Der Gewinn an Flexibilität führt jedoch zu einem höheren Bedarf an präzisen, pfadbasierten Abfragen. Verschachtelte Objektstrukturen und Arrays müssen gezielt adressiert werden. Teilbäume sollen sich selektiv auslesen lassen, und Werte sollen reproduzierbar extrahiert werden – ohne die gesamte Struktur manuell durchlaufen zu müssen.

Viele relationale Datenbanksysteme haben inzwischen spezielle Operatoren und Funktionen zur Abfrage von JSON-Daten integriert; teilweise steht sogar eine JSONPath-ähnliche Abfragesprache zur Verfügung. In YottaDB existiert eine solche Auswertungsfunktionalität bislang nicht. Zwar lassen sich JSON-ähnliche Strukturen als hierarchische Globals modellieren, was eine effiziente Speicherung ermöglicht, jedoch fehlt ein deklaratives und standardnahes Pfadkonzept. In der Praxis führt dies zu projektindividuellen Traversierungsroutinen, uneinheitlichen Konventionen und erhöhter Fehleranfälligkeit. Zudem erschwert die daraus resultierende Heterogenität die Wiederverwendbarkeit von Code, die Vergleichbarkeit von Ergebnissen sowie eine systematische Testabdeckung.

Vor diesem Hintergrund adressiert die vorliegende Arbeit eine konkrete Lücke. Sie zielt nicht auf eine vollständige JSONPath-Implementierung, sondern verfolgt einen pragmatischen Ansatz, der kurzfristig nutzbaren Mehrwert schafft und langfristig ausbaufähig bleibt. Ein CLI-Werkzeug nimmt JSONPath-Ausdrücke entgegen und übersetzt die Sprache in YottaDB-Referenzen. Die resultierende Abfrage wird lesend ausgeführt, der gefundene Wert in standardisierten Formaten ausgegeben. Damit wird die wiederkehrende Aufgabe, Werte aus verschachtelten Strukturen punktgenau zu extrahieren, konsistent und werkzeuggestützt gelöst.

Der Fokus auf ein CLI-Werkzeug ist durch seine Anschlussfähigkeit begründet. Es lässt sich unkompliziert in Skripte, Build-Pipelines und bestehende Tools integrieren; es eignet sich für schnelle Ad-hoc-Abfragen ebenso wie für

wiederholbare Testszenarien. Zugleich zwingt die klare I/O-Schnittstelle zu wohldefinierten Fehlermeldungen, stabilen Exit-Codes und nachvollziehbaren Semantiken, was die Gebrauchstauglichkeit erhöht und die Grundlage für Automatisierung legt.

Die Arbeit versteht sich damit als Brücke zwischen der schemalosen Ablage und einem deklarativen Lesezugriff in YottaDB. Sie adressiert typische Anforderungen in datenintensiven Anwendungen, in denen Entwicklerinnen und Entwickler schnell, reproduzierbar und mit geringem Implementierungsaufwand auf strukturierte Inhalte zugreifen können. Gleichzeitig schafft sie eine belastbare Basis, auf der künftig fortgeschrittene JSONPath-Konstrukte schrittweise und kontrolliert ergänzt werden können.

1.2 Problemstellung und -abgrenzung

Die Ausgangslage ist dadurch geprägt, dass YottaDB bislang keine deklarative Auswertungsfunktionalität für JSON-Daten bereitstellt. Zwar lassen sich JSON-ähnliche Strukturen effizient als hierarchische Globals modellieren, doch fehlt ein standardnahes Pfadkonzept, das die gezielte Adressierung verschachtelter Strukturen ermöglicht.

Die zentrale Problemstellung besteht darin, ein Verfahren zu schaffen, mit dem JSON-Strukturen in YottaDB präzise, reproduzierbar und ohne hohen Implementierungsaufwand adressiert werden können. Damit verbunden sind mehrere Herausforderungen: Eingaben müssen syntaktisch und semantisch korrekt interpretiert werden, die Abbildung auf YottaDB-Referenzen hat eindeutig und deterministisch zu erfolgen, und Fehlerzustände sind klar zu kommunizieren. Darüber hinaus ist eine Schnittstelle erforderlich, die sich in bestehende Entwicklungs- und Betriebsumgebungen integrieren lässt und konsistente Interaktionen erlaubt.

Um den Kern der Arbeit klar abzugrenzen und überprüfbar zu gestalten, liegt der Schwerpunkt auf einer Grundimplementierung mit ausgewählten Basisfunktionen. Im Mittelpunkt stehen die zuverlässige Verarbeitung einfacher Pfadkonstrukte, deren eindeutige Übersetzung nach YottaDB sowie eine nachvollziehbare und robuste Interaktion über die Kommandozeile. Dadurch entsteht eine stabile Grundlage, auf der spätere Erweiterungen des Sprachumfangs aufbauen können.

Die Abgrenzung ermöglicht es, die eigentliche Fragestellung – die zuverlässige Adressierung und Abfrage von Daten in YottaDB anhand eines klar definierten Pfadmodells – gezielt und nachvollziehbar zu bearbeiten.

1.3 Ziel der Arbeit

Das Ziel der Arbeit ist die Konzeption und Implementierung eines CLI-Tools, das eine Pipeline aus *Scanner* → *Parser* → *AST* → *YDB-Translator* → *Ausgabe* realisiert, welche eine lesende Abfragen in YottaDB ermöglicht. Hierzu werden eine modulare Zugriffsschicht auf YottaDB, ein abstrakter Syntaxbaum zur internen Repräsentation von Pfaden und Übersetzungsregeln vom AST auf YottaDB-Referenzen entworfen und umgesetzt. Die Indexsemantik wird dabei eindeutig festgelegt, um konsistente Ergebnisse über alle Aufrufe hinweg zu gewährleisten.

Die Zielerreichung wird durch drei zentrale Aspekte bestimmt. Maßgeblich ist zunächst die *Korrektheit*, also ob die übersetzten Pfade die erwarteten Zielknoten und Werte zuverlässig adressieren und ob auch Negativfälle korrekt behandelt werden. Ergänzend spielt die *Fehlertoleranz* eine Rolle: Syntax- und Semantikfehler sollen durch aussagekräftige Meldungen und stabile Exit-Codes erkennbar und reproduzierbar sein. Schließlich trägt die *Gebrauchstauglichkeit* zur Bewertung bei, indem nachvollziehbare Kommando- und Ausgabeformate, sinnvolle Voreinstellungen sowie ein dokumentiertes Verhalten sicherstellen, dass das Werkzeug im praktischen Alltag verwendbar bleibt. Diese Kriterien sind so formuliert, dass sie den realen Einsatz in Entwicklungs- und Betriebsumgebungen widerspiegeln, ohne die spätere Erweiterbarkeit einzuschränken.

1.4 Vorgehen

Das methodische Vorgehen orientiert sich an einem schrittweisen, integrierten Entwicklungs- und Evaluationsprozess. Zunächst werden Anforderungen und repräsentative Use-Cases erhoben. Diese orientieren sich an typischen Leseabfragen in YottaDB und dienen dazu, die erwarteten Eingabe- und Ausgabeformate präzise zu bestimmen. Auf dieser Basis entsteht eine Sprach-Vorarbeit: Tokens und Grammatikregeln für die definierte JSONPath-Teilmenge werden mit Flex und Bison spezifiziert und in einen Parser überführt, der einen wohldefinierten AST erzeugt. Der AST dient als stabile Schnittstelle zwischen Eingabe und Datenbankschicht; er ermöglicht Debug-Unterstützung, etwa durch einen Pretty-Printer, und erleichtert die systematische Ableitung von Übersetzungsregeln.

Im nächsten Schritt wird ein YottaDB-Translator implementiert, der die AST-Knoten deterministisch auf Referenzen im Global-Baum abbildet. Besonderes Augenmerk liegt auf klaren Entscheidungen zur Pfadsemantik, etwa dem Umgang mit fehlenden Schlüsseln, Typkonflikten oder ungültigen Indizes. Parallel entsteht eine Zugriffsschicht, die das lesende Ausführen der übersetzten Pfade übernimmt und Fehlersituationen konsistent behandelt. Die CLI integriert diese Komponenten zu einem geschlossenen Werkzeug mit stabilen Exit-Codes und gut dokumentierter Interaktion.

Die Qualitätssicherung erfolgt entlang zweier Linien. Erstens werden Funktionsnachweise durch Referenzfälle erbracht, die typische und grenzwertige Pfade abdecken und erwartete Zielwerte spezifizieren; Negativtests prüfen die Robustheit der Fehlermeldungen. Zweitens liefern Basismessungen einen Eindruck von der Laufzeit unter realistischen, punktuellen Lese-Szenarien. Abschließend werden potenzielle Erweiterungen skizziert.

2 Grundlagen

Dieses Kapitel vermittelt die theoretischen und technischen Grundlagen, die das Verständnis für die Umsetzung eines JSONPath-basierten Abfragesystems in YottaDB ermöglichen. Behandelt werden zunächst zentrale Konzepte wie JSON und JSONPath, sowie das Datenmodell von YDB. Darauf aufbauend werden das Mapping zwischen beiden Welten, ein durchgängiges Referenzbeispiel sowie grundlegende Aspekte der Parsertechnologie und des Fehlermodells vorgestellt. Abschließend werden die für die Arbeit relevanten Prinzipien der Kommandozeileninteraktion skizziert.

2.1 JSON – Format für strukturierte Daten

JSON ist ein textbasiertes Datenformat zur Darstellung strukturierter Daten. Es wurde ursprünglich für die Datenübertragung zwischen Webanwendungen entwickelt, hat sich jedoch mittlerweile als Standardformat in vielen Bereichen etabliert – insbesondere bei der Programmierung von Web-APIs und in NoSQL-Datenbanken.

Das JSON-Format zeichnet sich durch seine einfache Syntax, gute Lesbarkeit und hohe Flexibilität aus. Es unterstützt verschiedene Datentypen, darunter Zeichenketten (Strings), Zahlen, Wahrheitswerte (Booleans), Nullwerte sowie strukturierte Typen wie Objekte und Arrays. Die Daten werden in einer hierarchischen Struktur organisiert, was eine verschachtelte Darstellung komplexer Datenmodelle ermöglicht [1].

Grundlegende Struktur von JSON-Daten

Als laufendes Beispiel dient das JSON-Dokument in Listing 2.2 (Abschnitt 2.5). Das Wurzelelement ist ein Objekt `store` mit zwei Strukturen: einem Array `book` und einem Objekt `bicycle`. Jedes Buchobjekt enthält `category`, `author`, `title`, `price`; `bicycle` umfasst `color` und `price`.

Für die Arbeit mit JSON-Daten sind zwei Eigenschaften besonders relevant:

- **Hierarchie:** JSON erlaubt die Verschachtelung von Objekten und Arrays, wodurch komplexe und hierarchisch aufgebaute Datenstrukturen entstehen können.
- **Strukturfreiheit:** JSON-Dokumente sind schemalos, das heißt, sie haben im Gegensatz zu relationalen Datenbanken keine feste Struktur; Aufbau und Umfang können variieren.

Diese Eigenschaften ermöglichen flexible Datenmodelle, stellen jedoch zugleich spezifische Anforderungen an Abfragesprachen und Datenbanksysteme, wenn Systeme wie YottaDB nicht nativ JSON verarbeiten. Zur gezielten Auswertung werden Ausdruckssprachen eingesetzt; eine verbreitete Sprache ist *JSONPath* (Abschnitt 2.2).

2.2 JSONPath – Pfadbasierte Abfragesprache

JSONPath ist eine deklarative Abfragesprache zur Navigation in verschachtelten JSON-Dokumenten. Sie erlaubt es, bestimmte Werte oder Teilstrukturen präzise über kompakte Pfadausdrücke anzusprechen, ohne die gesamte Datenstruktur manuell durchlaufen zu müssen.

2.2.1 Standardisierung und Varianten

JSONPath wurde 2023 erstmals formell standardisiert im RFC 9535 [7]. Zuvor existierten mehrere voneinander abweichende Implementierungsvarianten (z. B. Jayway JSONPath [8], JSONPath Plus [2]). Die in dieser Arbeit verwendete Grammatik orientiert sich an der Spezifikation des RFC 9535.

2.2.2 Struktureller Aufbau von JSONPath-Ausdrücken

Ein JSONPath-Ausdruck wird formal als Folge von *Segmenten* definiert, die nacheinander ausgewertet werden und gemeinsam einen Pfad durch die JSON-Struktur bilden. Jedes Segment verfeinert die Adressierung und kann entweder auf ein Objektfeld oder ein Arrayelement verweisen. Damit ergibt sich eine schrittweise Navigation durch die hierarchische Struktur eines Dokuments [7].

In Abbildung 2.1 ist der Ausdruck `$.store.book[0].title` dargestellt. Er veranschaulicht den Aufbau aus dem Wurzelsymbol, zwei Namenssegmenten, einem Indexsegment und einem abschließenden Namenssegment.

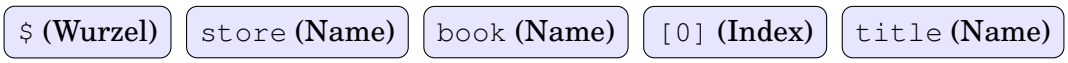

Abbildung 2.1: Struktureller Aufbau eines JSONPath-Ausdrucks

Wurzelsymbol Jeder JSONPath-Ausdruck beginnt mit dem Symbol `$`, das das Dokument als Ganzes referenziert. Dieses Wurzelsymbol bildet den Ausgangspunkt für alle weiteren Zugriffsschritte und stellt sicher, dass Pfade stets in einem klar definierten Kontext beginnen.

Segmentfolge Nach dem Wurzelsymbol folgt eine geordnete Abfolge von Segmenten. Jedes Segment repräsentiert einen Zugriffsschritt, der die Adressierung weiter verfeinert. Dabei wird zwischen Zugriffen auf Objektfelder und Zugriffen auf Arrayelemente unterschieden. Durch die Sequenz dieser Segmente entsteht eine eindeutige Navigation innerhalb der verschachtelten Struktur.

Selektoren Segmente können durch unterschiedliche Selektoren spezifiziert werden. Ein *Namens-Selektor* wählt ein Feld anhand seines Schlüssels, typischerweise in Punktnotation (`.name`) oder Klammernotation (`["name"]`). Ein *Index-Selektor* adressiert ein bestimmtes Arrayelement durch seinen numerischen Index (`[n]`). Diese Selektoren definieren die elementaren Zugriffsoperationen und erlauben es, sowohl Objektstrukturen als auch Arrays gezielt anzusprechen.

Hierarchie Die Segmentfolge ist nicht beliebig, sondern bildet eine strikt hierarchische Struktur ab. Jedes Segment baut auf dem vorherigen auf, sodass eine klare Eltern-Kind-Beziehung entsteht. Im Beispiel `$.store.book[0].title` bedeutet dies: vom Dokument (`$`) wird zunächst das Feld `store` ausgewählt, anschließend das Feld `book`, dann das erste Element des Arrays und schließlich dessen Feld `title`. Auf diese Weise wird ein Pfad durch die verschachtelte Struktur präzise festgelegt.

Bedeutung Der strukturelle Aufbau von JSONPath-Ausdrücken ermöglicht eine deklarative Beschreibung von Zugriffen, die unabhängig von einer programmatischen Traversierung ist. Statt die Struktur manuell zu durchlaufen, kann der gewünschte Knoten durch eine kompakte und formale Pfadangabe direkt adressiert werden.

2.2.3 Syntaxelemente von JSONPath

Aufbauend auf der Struktur definiert die Spezifikation von JSONPath verschiedene Ausdrucksformen, die unterschiedliche Zugriffe auf ein Dokument ermöglichen [7]. Die wichtigsten lassen sich in vier Kategorien einordnen:

- **Wurzelsymbol:** Jeder Ausdruck beginnt mit diesem Symbol, das das gesamte Dokument repräsentiert und als Ausgangspunkt für die Navigation dient.
- **Feldzugriffe:** Objektfelder können in Punkt-Notation oder in Klammer-Notation adressiert werden. Die Klammer-Variante wird insbesondere bei Sonderzeichen oder Leerzeichen im Feldnamen verwendet.
- **Array-Zugriffe:** Arrayelemente lassen sich über numerische Indizes ansprechen. Die Spezifikation erlaubt dabei auch negative Indizes, Vereinigungen mehrerer Positionen sowie Slices, um Teilbereiche auszuwählen.
- **Erweiterte Selektoren:** Dazu gehören Wildcards zur Auswahl aller Kinder oder Arrayelemente, der rekursive Deszent für die Suche über mehrere Tiefenebenen hinweg sowie Filterausdrücke zur Einschränkung der Ergebnismenge anhand boolescher Bedingungen.

Tabelle 2.1 fasst diese Syntaxelemente in kompakter Form zusammen.

Tabelle 2.1: JSONPath-Syntaxelemente

Syntaxelement	**Beschreibung**
`$`	Wurzel des JSON-Dokuments
`.name`	Feldzugriff (Punkt-Notation)
`["name"]`	Feldzugriff (Klammer-Notation, z. B. Sonderzeichen)
`[n]`	Array-Element per numerischem Index
`*`, `[*]`	Wildcards (alle Kinder/Elemente)
`[start:end[:step]]`	Array-Slice
`..`, `$..name`	Rekursiver Deszent
`[?<Bedingung>]`	Filterausdruck
`[-1]`, `[0,2]`	Negative Indizes, Union

2.2.4 Beispiele

Alle nachfolgenden Beispiele beziehen sich auf das Referenzbeispiel in Listing 2.2 (vgl. Abschnitt 2.5) und beschränken sich auf die im Teilumfang unterstützten Konstrukte.

Tabelle 2.2: Beispiele für JSONPath-Ausdrücke im implementierten Teilumfang

JSONPath-Ausdruck	**Ziel - Zugriff auf ...**
`$.store.bicycle.color`	das Feld `color` von `bicycle`
`$.store.bicycle.price`	das Feld `price` von `bicycle`
`$.store.book[0].author`	den Autor des ersten Buches
`$.store.book[1].title`	den Titel des zweiten Buches
`$["store"]["bicycle"]["price"]`	den Preis des Fahrrads

2.3 YottaDB – Überblick

YottaDB ist ein transaktionales, schemaloses Datenbanksystem auf Basis der M-Technologie (MUMPS). Es kombiniert einen persistenten, hierarchischen Key–Value-Speicher (*Globals*) mit Unterstützung für Atomicity, Consistency, Isolation, Durability (ACID)-konforme Transaktionen und eignet sich dadurch für Anwendungen, die große Datenmengen mit geringer Latenz verarbeiten müssen.

Das Datenmodell ist baumartig organisiert und erlaubt eine flexible, schemafreie Strukturierung. Neben der direkten Arbeit in der M-Programmiersprache stehen Anbindungen für verschiedene Programmiersprachen wie C, Go oder Python zur Verfügung [9].

Die für diese Arbeit relevanten Eigenschaften von YottaDB lassen sich wie folgt zusammenfassen:

- **Schemafreiheit:** Hierarchischer Key–Value-Speicher ohne vordefiniertes Schema; unterstützt flexible und evolvierende Datenmodelle.
- **Transaktionen:** ACID-konforme Transaktionen mit garantierter Dauerhaftigkeit und crash-sicherer Persistenz.
- **Performanz:** Sehr schnelle Punktzugriffe bei niedrigen Latenzen sowie effiziente Parallelisierung.
- **APIs:** Programmierschnittstellen für verschiedene Sprachen, geeignet für Automatisierung und Batch-Verarbeitung.

Da YottaDB, JSON und JSONPath nicht nativ unterstützt, übersetzt diese Arbeit die in Abschnitt 2.4 definierten JSONPath-Konstrukte in YottaDB-Referenzen.

2.3.1 Globals und Subskripte

Das YottaDB-Datenmodell basiert auf hierarchischen Globals mit Subskripten [9]. Ein Zugriff auf ein Global besteht aus dem Globalnamen und einer geordneten Liste von *Subskripten*:

$$\hat{}G(s_1, \ldots, s_n)$$

Jedes Subskript wird entweder als Zahl oder als String interpretiert. Blattknoten speichern Werte, während innere Knoten der hierarchischen Strukturierung dienen.

Subskript-Typisierung In YottaDB werden Subskripte kanonisiert, sodass numerische Strings und Zahlen denselben Knoten adressieren können (z.,B. `^g("1")` und `^g(1)`). Für das Mapping aus JSONPath bedeutet dies: Obwohl `["1"]` (String) und `[1]` (Zahl) semantisch verschieden sind, können sie in YottaDB ohne zusätzliche Typkodierung nicht eindeutig unterschieden werden und somit kollidieren (vgl. Abschnitt 2.4).

Ablage von JSON-ähnlichen Strukturen Für JSON-ähnliche Daten bietet es sich an, pro logischer Sammlung ein eigenes Global (z.,B. `^store`) zu verwenden. Objektfelder werden dabei als String-Subskripte, Arrayelemente als numerische Subskripte repräsentiert. Das notwendige Index-Mapping von JSON (0-basiert) auf YottaDB (1-basiert) wird im Mapping-Abschnitt spezifiziert (vgl. Abschnitt 2.4). Eine schematische Ablage des Referenzbeispiels ist in Listing 2.3 dargestellt.

Setzen und Lesen eines Datensatzes Im M-Interpreter von YottaDB lassen sich Knoten direkt adressieren:

```
YDB> SET ^store("bicycle","color") = "red"
YDB> WRITE ^store("bicycle","color")
red
```

Listing 2.1: Setzen und Lesen eines Werts in YottaDB

Das Beispiel zeigt eine klassische Key–Value-Struktur: Unter dem Schlüssel `^store("bicycle","color")` wird der Wert „red“ abgelegt und später über denselben Schlüssel wieder gelesen.

Diese Funktionen sowie `$DATA` und `$GET` werden von YottaDB bereitgestellt und in dieser Arbeit gemäß der offiziellen Dokumentation genutzt [9].

Datenflags mit `$DATA` Die Existenz- und Struktureigenschaften eines Knotens können in YottaDB über die Funktion `$DATA` ermittelt werden:

0	Knoten existiert nicht
1	Knoten enthält einen Wert
10	Knoten besitzt ausschließlich Kindknoten
11	Knoten enthält einen Wert *und* besitzt Kindknoten

Die Rückgabecodes differenzieren somit explizit zwischen Skalarwerten und strukturierten (Teil-)Bäumen und bilden die Grundlage für eine deterministische Ausgabe- und Fehlersemantik.

Kanonische Traversierung Für die Auflistung von Kindknoten stellt YottaDB die Kombination aus `$NA` (kanonische Referenz) und `$QUERY` (Iteration über Nachfolger) bereit. Diese Arbeit nutzt jedoch ausschließlich `$DATA` und `$GET`, eine Traversierung von Kindknoten ist nicht implementiert.

2.3.2 Routinen

Ein zentrales Konzept in YottaDB sind *Routinen*. Dabei handelt es sich um Programmeinheiten, die in der M-Sprache (MUMPS) formuliert und im Datenbankkontext ausgeführt werden. Routinen sind typischerweise in Dateien mit der Endung `.m` abgelegt, werden von YottaDB kompiliert und anschließend in optimierter Form bereitgestellt. Sie enthalten Prozeduren, Funktionsdefinitionen und Hilfsprogramme für den Datenbankzugriff.

Routinen können direkt innerhalb der YottaDB-Umgebung gestartet oder aus anderen Routinen heraus aufgerufen werden. Sie bilden die Basis für modulare Programmierung in M und ermöglichen die Kapselung wiederverwendbarer Logik, etwa zur Datenverarbeitung, zur Abbildung von Geschäftsregeln oder zur Bereitstellung von Schnittstellenfunktionen. Da Routinen Werte zurückgeben können, lassen sie sich wie klassische Funktionsaufrufe in komplexere Abläufe integrieren.

Neben vordefinierten Routinen stellt YottaDB Mechanismen bereit, eigene Routinen in die Laufzeitumgebung einzubinden. Dies wird unter anderem genutzt, um Brücken zwischen externen Programmen und den in der Datenbank gespeicherten Daten zu realisieren. Auf diese Weise lassen sich Datenbankoperationen automatisieren und systematisch erweitern.

2.4 Mapping JSONPath → YottaDB

Das Mapping bildet die Verbindung zwischen JSONPath-Ausdrücken und der baumartigen Repräsentation in YottaDB. Es legt fest, wie Pfade auf die hierarchische Struktur von Globals übertragen werden. Ziel ist eine eindeutige und konsistente Übersetzung, die ausschließlich lesend erfolgt und den Typ der Subskripte unverändert beibehält. Eine automatische Umwandlung zwischen numerischen und stringbasierten Subskripten findet nicht statt.

2.4.1 Voraussetzungen und Notation

Als Einstiegspunkt dient ein *Wurzel-Global* G, der die oberste Ebene der Datenbankstruktur markiert. Die in Abschnitt 2.2 beschriebenen Segmente werden nach definierten Regeln auf Subskripte in YottaDB abgebildet. Das Wurzelsymbol `$` selbst wird dabei nicht materialisiert, sondern verweist unmittelbar auf den angegebenen Wurzel-Global G.

Ein Ausdruck, der ausschließlich aus dem Wurzelsymbol besteht, entspricht `^G` ohne weitere Subskripte. Feldnamen mit Sonderzeichen werden in Klammer-Notation angegeben, wodurch syntaktische Eindeutigkeit sichergestellt wird.

2.4.2 Basismapping

Die Übersetzung wird durch eine Funktion $\mathcal{T}_G$ beschrieben, die einen JSONPath-Ausdruck auf eine YottaDB-Referenz `^G`$(\cdot)$ abbildet:

$$\mathcal{T}_G(\$\, s_1 \ldots s_k) \;=\; \texttt{\^{}G}(\sigma_1, \ldots, \sigma_k)$$

mit den Regeln

$$\sigma_i = \begin{cases} \texttt{"name"} & \text{für Feldzugriffe in Punkt- oder Klammer-Notation,} \\ n+1 & \text{für Array-Indizes } [n],\ n \in \mathbb{N}_0, \\ m+1,\, n+1, \ldots & \text{für Vereinigungen (Union) mehrerer Indizes,} \\ |A| - n & \text{für negative Indizes, wobei } |A| \text{ die Länge des Arrays ist,} \\ * & \text{für Wildcards, die alle Kinderknoten adressieren.} \end{cases}$$

Beispiele (für G=`store`):

JSONPath	YottaDB-Referenz
`$.bicycle.color`	`^store("bicycle","color")`
`$.book[0].author`	`^store("book",1,"author")`
`$["bicycle"]["price"]`	`^store("bicycle","price")`
`$.book[0,2].title`	`^store("book",1,"title");`
	`^store("book",3,"title")`
`$.book[*].author`	`^store("book",*,"author")`

Hinweis: Das führende Segment `"store"` wird nicht als Subskript materialisiert, da der Wurzel-Global $G =$ `store` separat parametrisiert ist.

2.4.3 Typkodierung von Subskripten

In JSON sind String- und Zahlensubskripte semantisch eindeutig voneinander unterschieden: `["1"]` bezeichnet den String `"1"`, während `[1]` die Zahl 1 darstellt. Zugelassen sind ausschließlich nicht-negative ganzzahlige Dezimalliterale; negative oder nicht-ganzzahlige Indizes sowie gemischte Formen sind nicht Teil der Spezifikation.

YottaDB hingegen führt eine Kanonisierung von Subskripten durch, sodass numerische Strings und Zahlen intern als gleichwertig behandelt werden. Daher verweisen `^g("1")` und `^g(1)` auf denselben Knoten. Ohne eine explizite Typkodierung lassen sich in JSON zulässige Felder wie `"1"` und Arrayindizes `1` in YottaDB nicht eindeutig voneinander unterscheiden.

Mögliche Erweiterung: Typkodierung

Zur eindeutigen Trennung von Objektfeldern und Arrayindizes wäre eine *Typkodierung* der Subskripte denkbar. Dabei würde jedes Segment s_i auf ein Paar aus Typpräfix und Wert abgebildet, sodass die resultierende Referenz diese Paare zu einer Sequenz flacht:

$$\mathcal{T}_G^{\tau}(\$\, s_1 \ldots s_k) = \texttt{\^{}G}(\tau_1, v_1, \ldots, \tau_k, v_k).$$

Die Präfixe `"k"` (*key*) und `"i"` (*index*) könnten den Segmenttyp explizit kodieren. Damit ließen sich Konstrukte wie `["1"]` (Feld) und `[1]` (Index) eindeutig unterscheiden, unabhängig von der Kanonisierung in YottaDB.

2.5 Referenz Beispiel

Zur Veranschaulichung wird im Folgenden ein durchgängiges Beispiel verwendet. Es dient als Referenzfall für die Darstellung von JSONPath-Ausdrücken, deren Abbildung auf YottaDB-Referenzen sowie die Interpretation der Ergebnisse. Das Beispiel orientiert sich an einem typischen Shop-Datensatz, der Bücher, ein Fahrrad sowie Informationen zu einem Lager umfasst.

JSON-Dokument

```
{
  "store": {
    "book": [
      { "category": "fiction", "author": "Herman Melville",
        "title": "Moby Dick", "price": 8.99 },
      { "category": "fiction", "author": "J. R. R. Tolkien",
        "title": "The Lord of the Rings", "price": 22.99 }
    ],
    "bicycle": { "color": "red", "price": 399 }
  },
  "warehouse": {"location": "Berlin", "capacity": 1200}
}
```

Listing 2.2: Beispiel eines JSON-Dokuments

Äquivalente YottaDB-Ablage (`^store` und `^warehouse`)

```
^store("bicycle","color")  = "red"
^store("bicycle","price")  = 399

^store("book",1,"author")  = "Herman Melville"
^store("book",1,"title")   = "Moby Dick"
^store("book",1,"category")= "fiction"
^store("book",1,"price")   = 8.99

^store("book",2,"author")  = "J. R. R. Tolkien"
^store("book",2,"title")   = "The Lord of the Rings"
^store("book",2,"category")= "fiction"
^store("book",2,"price")   = 22.99

^warehouse("location")     = "Berlin"
^warehouse("capacity")     = 1200
```

Listing 2.3: Schematische Ablage in YottaDB

Mapping-Beispiele

Tabelle 2.3: Beispiele: JSONPath → YDB-Referenz → erwarteter Wert

JSONPath	YDB-Referenz	Wert
`$.bicycle.color`	`^store("bicycle","color")`	`"red"`
`$.bicycle.price`	`^store("bicycle","price")`	`399`
`$.book[0].author`	`^store("book",1,"author")`	`"Herman Melville"`
`$.book[0].price`	`^store("book",1,"price")`	`8.99`
`$.book[1].title`	`^store("book",2,"title")`	`"The ... Rings"`
`$.book[2].title`	`^store("book",3,"title")`	*nicht vorhanden*

Hinweis: Der Wurzel-Global `^store` wird separat parametrisiert. Das führende Segment `"store"` muss daher im JSONPath-Ausdruck nicht als Subskript materialisiert werden (vgl. Abschnitt 2.4).

2.6 Parser

Die Entwicklung eines Parsers erfordert grundlegende Konzepte aus der Theorie formaler Sprachen. Im Folgenden werden zwei zentrale Aspekte eingeführt: die Chomsky-Hierarchie zur Einordnung von Sprachklassen sowie die Extended Backus-Naur Form (EBNF) zur formalen Definition der Syntax.

2.6.1 Formale Sprachen und Chomsky-Hierarchie

Formale Sprachen werden über Mengen von Regeln definiert, die festlegen, welche Zeichenketten zu einer Sprache gehören. Eine zentrale Klassifikation liefert die Chomsky-Hierarchie. Diese klassifiziert formale Sprachen in vier Haupttypen, basierend auf der Ausdrucksstärke ihrer jeweiligen Grammatiken. Diese Einteilung ist insbesondere im Compilerbau von Bedeutung, da sie Rückschlüsse darauf erlaubt, welche Parsermechanismen für eine bestimmte Sprache geeignet sind.

Die Abbildung 2.2 veranschaulicht die hierarchische Einordnung der vier Sprachtypen in Form eines Mengendiagramms.

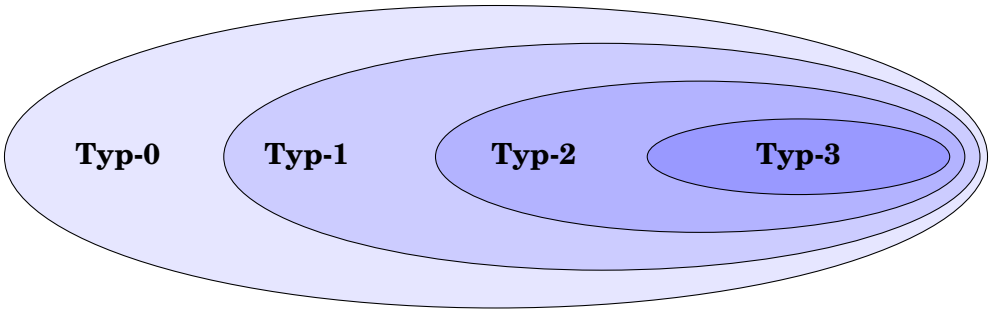

Abbildung 2.2: Verschachtelte Darstellung der Chomsky-Hierarchie

- **Typ-3 – Reguläre Sprachen:** Sie können durch endliche Automaten erkannt werden. Beispiele sind einfache Suchmuster oder reguläre Ausdrücke.
- **Typ-2 – Kontextfreie Sprachen:** Sie werden durch Kellerautomaten (Push-down Automaten) verarbeitet. Viele Programmiersprachen gehören zu dieser Klasse.
- **Typ-1 – Kontextsensitive Sprachen:** Sie erfordern eine Turingmaschine mit beschränktem Arbeitsspeicher. In der Praxis spielen sie eine eher untergeordnete Rolle.
- **Typ-0 – Rekursiv aufzählbare Sprachen:** Sie sind die allgemeinste Klasse formaler Sprachen und werden durch Turingmaschinen erkannt.

Die Syntax von JSONPath lässt sich im Allgemeinen als kontextfreie Sprache (Typ-2) modellieren, da sie rekursive Strukturen wie Punktnotation, Array-Zugriffe und verschachtelte Filterausdrücke umfasst. Solche Sprachen können von Parsergeneratoren wie *Yacc* oder *Bison* [4] verarbeitet werden, die auf LR-Parsing-Verfahren basieren. Allgemein gilt: Parser im Compilerbau operieren typischerweise auf kontextfreien Grammatiken, da diese eine gute Balance zwischen Ausdrucksstärke und algorithmischer Beherrschbarkeit bieten.

Syntaxdefinition mit EBNF

Die *Erweiterte Backus–Naur-Form* (EBNF) ist eine formale Notation zur Beschreibung kontextfreier Syntax. Sie bietet kompakte Konstrukte für Wiederholung ({}), Option ([ ]) und Gruppierung (()). Im Folgenden ist ein vereinfachter Ausschnitt einer EBNF-Grammatik für JSONPath dargestellt; er deckt genau die Grundmuster ab, die im weiteren Verlauf als Beispiel verwendet werden:

```
1 Path    ::= '$' (Segment)*
2 Segment ::= '.' Name | '["' Name '"]' | '[' Index ']'
3 Name    ::= [a-zA-Z_][a-zA-Z0-9_]*
4 Index   ::= Digit+
5 Digit   ::= '0' | '1' | ... | '9'
```

Listing 2.4: Beispiel einer vereinfachten EBNF-Grammatik für JSONPath

Diese Grammatik beschreibt Pfade, die an der Wurzel (`$`) beginnen und aus Segmenten in Punktnotation (`.Name`) sowie Arrayzugriffen in Klammernotation (`[Index]`) bestehen. Auch String-Schlüssel (`["name"]`) sind abbildbar. Erweiterungen wie Wildcards, Filter, Slices oder rekursiver Deszent sind bewusst nicht enthalten, können aber auf dieser Basis ergänzt werden.

Nach dieser Grammatik gehören Ausdrücke wie `$.bicycle.price` oder `$.book[0].title` zur Sprache.

2.6.2 Lexikalische und syntaktische Analyse

Ein typisches Parsersystem besteht aus zwei Stufen:

- **Lexikalische Analyse (Lexing):** Die Eingabezeichenkette wird in Tokens zerlegt, also grundlegende Einheiten wie Zahlen, Operatoren oder Bezeichner.
- **Syntaktische Analyse (Parsing):** Der Parser prüft die Folge der Tokens anhand der Grammatik und erzeugt eine strukturierte Repräsentation, typischerweise einen Abstract Syntax Tree (AST).

Der Einsatz von Flex und Bison [5, 4] ermöglicht dabei eine modulare, erweiterbare und wartbare Architektur: Die Trennung von lexikalischer Analyse, syntaktischer Strukturierung und späterer Auswertung erlaubt es, die einzelnen Komponenten unabhängig weiterzuentwickeln.

2.6.3 Abstract Syntax Tree (AST)

Der AST ist eine Baumstruktur, die die syntaktische Organisation der Eingabe abbildet, ohne unnötige syntaktische Details wie Klammern oder Trennzeichen beizubehalten. Anhand eines JSONPath-Ausdrucks wird sichtbar, wie die Hierarchie der Pfadsegmente in eine Baumstruktur überführt wird.

Am Beispiel des Ausdrucks `$.book[0].title` ergibt sich der in Abbildung 2.3 dargestellte AST:

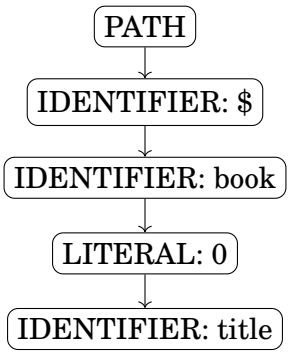

Abbildung 2.3: Beispiel AST für einen JSONPath-Ausdruck

Der AST abstrahiert also von der linearen Eingabe und macht die hierarchische Struktur der Pfadsegmente explizit sichtbar. Dies bildet die Grundlage für die spätere semantische Auswertung.

2.6.4 Besonderheiten des Parsers

- Es werden ausschließlich nichtnegative ganzzahlige Literale akzeptiert; negative, nicht-ganzzahlige oder gemischte Formen sind ausgeschlossen.
- Es findet keine Keyword-Reservierung statt. Auch Zeichenketten wie `true`, `false` oder `null` können als Feldnamen verwendet werden, solange keine Filterausdrücke vorkommen.
- Fehlerhafte Konstrukte werden vom Parser nicht akzeptiert und führen zu klaren Fehlermeldungen.

2.7 Fehlermodell

Zur Erwartungssteuerung und Testbarkeit unterscheidet die Implementierung zwischen *Syntaxfehlern*, *semantischen Fehlern* und *Systemfehlern*. Jede Klasse wird mit einem reproduzierbaren Exit-Code signalisiert.

Exit-Code	Bedeutung
`EX_OK (0)`	Erfolg
`EX_PARSE (2)`	Syntaxfehler oder nicht unterstützte Konstrukte
`EX_NOTFOUND (3)`	Pfad existiert nicht (`$DATA=0`)
`EX_YDB (<0)`	Fehler beim Zugriff auf YottaDB (z. B. Bridge-Datei fehlt)

Ausgabe- und Meldekanäle Erfolgswerte erscheinen auf `stdout`, Fehlermeldungen auf `stderr`.

Syntaxfehler (Lexing/Parsing) Unerwartete Zeichen oder unvollständige Strukturen (z. B. `$.book[`) führen zu einem Abbruch mit `EX_PARSE`. Auch Konstrukte außerhalb des definierten Teilumfangs werden vom Parser gezielt erkannt und mit spezifischen Meldungen abgewiesen (z. B. „Rekursiver Deszent nicht unterstützt (n. u.)“, „Wildcard n. u.“ sowie weitere Meldungen). Die Fehlermeldungen werden in `parser.y` über `yyerror` im `verbose`-Modus erzeugt. Eine Übersicht konkreter Negativfälle und der ausgegebenen Meldungen findet sich in Abschnitt 7 (Tabelle 7.2).

Semantische Fehler (Auswertung) Ein syntaktisch korrekter Pfad kann semantisch „ins Leere“ führen, etwa wenn ein adressierter Knoten nicht existiert (`$DATA=0`). In solchen Fällen beendet das CLI mit `EX_NOTFOUND` und der Meldung „Pfad nicht gefunden.“ Eine differenzierte Behandlung von Typkonflikten (z. B. Index auf ein Objekt) oder eine feingranulare Teilpfadangabe ist in der aktuellen Implementierung nicht vorgesehen.

Systemfehler (Laufzeit/Umgebung) Fehler bei der YottaDB-Interaktion werden getrennt von Eingabefehlern behandelt und führen zu einem negativen Exitcode. Dieser Wert entstammt nicht der definierten Exitcode-Tabelle, sondern wird direkt aus Rückgabewerten der POSIX- bzw. YottaDB-Aufrufe durchgereicht. Diese Fehlerklasse betrifft Betriebsbedingungen, nicht die Eingabesyntax.

3 Problemanalyse

Die Problemanalyse dient dazu, die Ausgangssituation, bestehende Defizite sowie die daraus abgeleiteten Anforderungen präzise zu beschreiben. Sie bildet damit die Grundlage für die Entwicklung eines geeigneten Konzepts und schärft zugleich die Risiken und Randbedingungen, die bei der Umsetzung zu berücksichtigen sind.

3.1 Ist-Zustand und Lücke in YottaDB

YottaDB stellt einen hierarchischen, schemalosen Key–Value-Speicher (*Globals*) bereit, besitzt jedoch keine native Auswertung von JSON oder JSONPath. In der Praxis führt dies zu individuell implementierten Traversierungsroutinen, uneinheitlichen Konventionen und einer erhöhten Fehleranfälligkeit.

Die fehlende Unterstützung für eine deklarative, standardisierte Pfadsprache erzeugt eine funktionale Lücke: JSON-Strukturen lassen sich zwar effizient speichern, aber nicht ohne weiteres präzise und reproduzierbar abfragen. Dies erschwert die Wiederverwendbarkeit von Code, die Vergleichbarkeit von Ergebnissen und eine systematische Testabdeckung.

3.2 Anforderungen und Randbedingungen

Aus der Analyse ergeben sich mehrere Anforderungen, die sowohl funktionale als auch qualitative Aspekte betreffen:

- **Funktionale Abdeckung:** Ein Kernumfang elementarer Zugriffskonstrukte (Feldzugriffe und Arrayindizes) genügt, um typische JSON-Datenstrukturen adressieren zu können. Eine vollständige Abdeckung aller in RFC 9535 definierten Konstrukte würde den Komplexitätsgrad deutlich erhöhen und ist für eine Grundimplementierung nicht erforderlich.
- **Index-Semantik:** JSON-Indizes sind 0-basiert, YottaDB-Subskripte 1-basiert. Eine konsistente Abbildung dieser beiden Modelle ist notwendig, um eindeutige Ergebnisse zu gewährleisten.

- **Lesender Zugriff:** Abfragen sollen den Datenbestand nicht verändern, sondern ausschließlich lesend arbeiten. Dadurch bleibt der Systemzustand unverändert und die Sicherheit erhöht.
- **Determinismus:** Übersetzungen müssen eindeutig, Fehlermeldungen klar und reproduzierbar sein. Nur so lassen sich Abfragen in Test- und Produktionsumgebungen zuverlässig einsetzen.
- **Umgebungsbedingungen:** Für die Integration in bestehende Systeme sind wohldefinierte Schnittstellen und reproduzierbare Ausführungsbedingungen erforderlich.

3.3 Alternativen und Abwägungen

Eine naheliegende Möglichkeit wäre die Nutzung bestehender Implementierungen von JSONPath. So stellt etwa PostgreSQL einen RFC-9535-konformen Parser bereit [10]. Dieser ist jedoch tief in die PostgreSQL-Engine integriert und nicht ohne erheblichen Aufwand isoliert nutzbar. Ein direkter Einsatz in Kombination mit YottaDB würde daher eine komplexe Portierung oder die Einführung einer zusätzlichen Datenbankebene erfordern.

Auch eigenständige Bibliotheken wie Jayway JSONPath (Java) [8] oder JSONPath Plus (JavaScript) [2] existieren. Diese sind jedoch auf ihre jeweiligen Ökosysteme zugeschnitten und weichen teils von der RFC-Spezifikation ab.

Die Analyse zeigt damit, dass eine an die Bedürfnisse von YottaDB angepasste Neuentwicklung den größten Handlungsspielraum bietet. Sie ermöglicht eine präzise Kontrolle über Grammatik, Fehlermodell und Mapping und schafft die Basis für eine konsistente und erweiterbare Lösung.

Die Entwicklung einer eigenen Grundimplementierung ist jedoch mit Risiken verbunden, insbesondere im Hinblick auf Funktionalität, Erweiterbarkeit und Betriebssicherheit. Wesentliche Punkte sind:

- **Subskript-Kanonisierung in YottaDB:** Numerische Strings und Zahlen können denselben Knoten adressieren (`^g("1")` vs. `^g(1)`). Ohne klare Unterscheidung drohen Mehrdeutigkeiten.
- **Umgebungsabhängigkeiten:** Fehler bei Konfiguration oder Laufzeitumgebung können die Ausführung blockieren.
- **Nicht unterstützte Syntax:** Konstrukte außerhalb des Kernumfangs dürfen nicht zu undefiniertem Verhalten führen.

4 Lösungskonzept

Das Lösungskonzept beschreibt die Architektur und das Sprachmodell des geplanten Werkzeugs. Ziel ist es, eine robuste und nachvollziehbare Verarbeitungskette von JSONPath-Ausdrücken bis zur Ausführung in YottaDB zu entwerfen. Dazu werden zentrale Komponenten, deren Zusammenspiel sowie die Prinzipien der Übersetzung und Fehlerbehandlung strukturiert vorgestellt.

4.1 Architekturübersicht

Die Architektur orientiert sich an einem klar geschichteten Modell, in dem jedes Modul eine wohldefinierte Aufgabe übernimmt. Tabelle 4.1 fasst die zentralen Bausteine zusammen.

Komponente	**Aufgabe im Entwurf**
Scanner	Identifiziert die elementaren Bestandteile eines Ausdrucks und zerlegt den Eingabestring in Tokens .
Parser	Setzt die Tokens zu einer strukturierten Darstellung zusammen und prüft die syntaktische Korrektheit der Eingabe.
AST	Stellt den Ausdruck als Baumstruktur dar und bildet die Grundlage für die weitere Übersetzung.
Mapping	Überträgt die abstrakte Struktur in eine formale Datenbankreferenz; sorgt für eine deterministische und eindeutige Abbildung.
Bridge	Stellt die Schnittstelle zur Datenbank her und dient als konzeptionelle Verbindung zwischen Pfadmodell und Speicherstruktur.
CLI	Koordiniert die Ausführung, bündelt Optionen und steuert die Ausgabe; ermöglicht die Einbettung in unterschiedliche Nutzungskontexte.
Fehlerbehandlung	Erkennung fehlerhafter Eingaben, Kennzeichnung nicht abbildbarer Konstrukte und systematische Rückmeldung an die Benutzungsoberfläche.

Tabelle 4.1: Architekturkomponenten und ihre konzeptionellen Aufgaben

4.2 Architekturpipeline

Die Verarbeitung folgt einer festen Abfolge: Vom Eingabestring gelangt die Anfrage über Scanner und Parser zu einer Baumdarstellung (AST), die durch das Mapping in eine YottaDB-Referenz übertragen wird. Anschließend stellt die Bridge die Verbindung zur Datenbank her und liefert Werte oder Marker zurück. Abbildung 4.1 veranschaulicht diesen Ablauf.

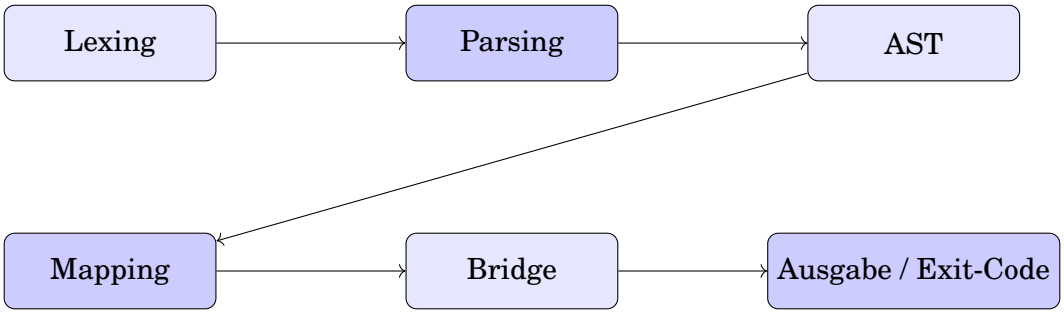

Abbildung 4.1: Pipeline vom JSONPath-String bis zur YottaDB-Ausgabe

4.3 Sprachmodell

Das Sprachmodell definiert die in der Grundimplementierung unterstützten Konstrukte von JSONPath und deren Abbildung in eine interne Struktur. Es stellt sicher, dass Abfragen reproduzierbar verarbeitet werden können und liefert einen stabilen Rahmen für spätere Erweiterungen.

Unterstützt werden fundamentale Ausdrucksformen: das Wurzelsymbol `$`, Feldzugriffe in Punkt- oder Klammernotation (z. B. `.name` oder `["name"]`) sowie Array-Indizes `[n]`. Für letztere wird die 0-basierte Notation von JSON auf die 1-basige Subskriptlogik von YottaDB abgebildet.

Die interne Repräsentation erfolgt über einen Abstrakten Syntaxbaum, der die Hierarchie der Pfadsegmente wiedergibt. Damit bleibt die Übersetzung sowohl für einfache Abfragen als auch für komplexere Strukturen konsistent.

4.4 Übersetzung zu YottaDB

Die Abbildung von JSONPath-Ausdrücken auf YottaDB erfolgt nach klar definierten Regeln, die eine konsistente und vorhersagbare Referenzierung gewährleisten:

- Das Wurzelsymbol `$` dient ausschließlich als Startpunkt und wird nicht als Subskript materialisiert.
- Objektfelder werden als String-Subskripte übernommen. Sie können sowohl in Punkt-Notation (`.name`) als auch in Klammer-Notation (`["name"]`) angegeben werden; die Klammer-Variante ermöglicht die Nutzung von Sonderzeichen oder Leerzeichen.
- Array-Indizes `[n]` werden systematisch von der 0-basierten JSON-Notation auf die 1-basige Subskriptlogik von YottaDB abgebildet.
- Ausdrucksbestandteile außerhalb des definierten Umfangs werden nicht in eine Referenz übersetzt, sondern als ungültig zurückgewiesen.

Diese Regeln schaffen eine eindeutige Verbindung zwischen der logischen Struktur von JSON-Dokumenten und der hierarchischen Organisation von Globals in YottaDB. Gleichzeitig wird sichergestellt, dass keine impliziten Typumwandlungen oder semantischen Mehrdeutigkeiten auftreten.

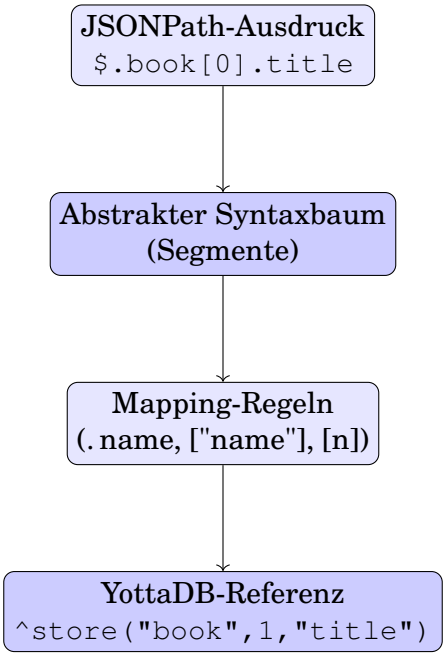

Abbildung 4.2: Übersetzung eines JSONPath-Ausdrucks auf eine YottaDB-Referenz

4.5 Zugriffsschicht

Da YottaDB keine native Auswertung von JSONPath-Ausdrücken bereitstellt, übernimmt eine Vermittlungsschicht die Ausführung der übersetzten Referenzen. Diese Brücke bildet die Schnittstelle zwischen der Pfadübersetzung und der Datenbank und sorgt dafür, dass Abfragen kontrolliert im Datenbankkontext ausgeführt werden.

Ein zentrales Element bilden dabei *Routinen* in der M-Sprache (MUMPS). Sie sind kompiliert in der Datenbankumgebung verfügbar und können direkt aufgerufen werden, um Daten zu lesen oder Statusinformationen (z. B. Existenz oder Struktur eines Knotens) zurückzugeben. Die Zugriffsschicht nutzt solche Routinen, um eine klare Trennung zwischen Abfrage-Logik und Datenbankinteraktion zu gewährleisten.

Die Entscheidung für diese Architektur folgt mehreren Anforderungen:

- **Determinismus:** Ergebnisse werden in einem wohldefinierten Format bereitgestellt und sind reproduzierbar.
- **Robustheit:** Der Datenbankzugriff ist gekapselt; Fehler können isoliert erkannt und behandelt werden.
- **Integrationsfähigkeit:** Routinen lassen sich aus verschiedenen Kontexten aufrufen, wodurch eine einfache Einbettung in Skripte oder Testumgebungen möglich ist.
- **Nachvollziehbarkeit:** Standardisierte Rückgaben erleichtern die Diagnose und schaffen Transparenz.

4.6 Fehlermanagement

Das Fehlermanagement ist integraler Bestandteil der Zugriffsschicht und unterscheidet zwischen klar abgegrenzten Kategorien:

- **Syntaxfehler:** treten auf, wenn ein Ausdruck fehlerhaft ist oder nicht zur unterstützten Teilsprache gehört; sie werden bereits in der Parsing-Phase erkannt und mit eindeutigen Meldungen zurückgegeben.
- **Navigationsfehler:** entstehen, wenn gültige Ausdrücke auf nicht vorhandene Knoten verweisen. Sie werden durch die Abfrage von Routinen festgestellt.
- **Systemfehler:** umfassen Störungen der Ausführungsumgebung, etwa fehlende Routinen, Probleme beim Zugriff auf die Datenbank oder Fehler in der Container-Infrastruktur.

Die klare Trennung dieser Fehlerarten stellt sicher, dass die Reaktion auf Probleme deterministisch bleibt: Syntaxfehler erfordern eine Korrektur des Eingabeausdrucks, Navigationsfehler liefern nachvollziehbare Hinweise auf fehlende Daten, und Systemfehler signalisieren technische Störungen. Damit bleibt das Verhalten des Gesamtsystems sowohl für Nutzer als auch für nachgelagerte Automatisierungen konsistent und vorhersagbar.

4.7 CLI-Design

Das Werkzeug ist als Kommandozeilenprogramm konzipiert und folgt den Prinzipien klarer und reproduzierbarer Ausgabekanäle:

- **Standardausgabe (`stdout`):** gibt ausschließlich die inhaltlichen Ergebnisse einer Abfrage zurück.
- **Fehlerausgabe (`stderr`):** dient für Fehlermeldungen und Diagnosetexte, um eine saubere Trennung von Nutzwerten und Metainformation zu gewährleisten.

Diese Trennung stellt sicher, dass das Werkzeug sowohl im interaktiven Betrieb als auch in automatisierten Skripten oder Build-Pipelines zuverlässig eingesetzt werden kann.

Darüber hinaus werden mehrere Modi und Optionen vorgesehen:

- **Quiet-Modus (`-q`):** unterdrückt ergänzende Texte und gibt ausschließlich den Wert der Abfrage aus; nützlich für Skripte.
- **AST-Ansicht (`-a`):** zeigt die interne Struktur des geparsten Ausdrucks in Form eines Syntaxbaums.
- **Referenzansicht (`-r`):** gibt die aus dem Ausdruck generierte YottaDB-Referenz aus.
- **Interaktivmodus:** ermöglicht die Eingabe mehrerer Ausdrücke in einer Sitzung; hilfreich für explorative Abfragen und Tests.
- **Hilfetext (`-h`):** fasst verfügbare Optionen und Rückgabecodes zusammen.

Ein zentrales Entwurfsziel ist die **Standardisierung der Exit-Codes**, um Fehlertypen klar abgrenzen zu können. Dazu gehören: `EX_OK` (erfolgreiche Abfrage), `EX_PARSE` (Syntaxfehler), `EX_NOTFOUND` (kein Treffer) sowie `EX_YDB`.

Insgesamt wird das Interface so gestaltet, dass es sowohl für schnelle Ad-hoc-Abfragen als auch für reproduzierbare Testszenarien geeignet ist.

4.8 Erweiterbarkeit

Ein zentrales Entwurfsprinzip des Werkzeugs ist die gezielte Erweiterbarkeit. Die klare Trennung von Scanner, Parser, AST, Übersetzer und Zugriffsschicht stellt sicher, dass zusätzliche Sprachkonstrukte von JSONPath schrittweise ergänzt werden können, ohne den bestehenden Kern zu destabilisieren.

Neue Selektoren oder Operationen können dabei isoliert in Grammatik und Mapping eingeführt werden, während die übrigen Komponenten unverändert bleiben. Auch die Zugriffsschicht ist so konzipiert, dass Erweiterungen der Pfadsprache unmittelbar in erweiterte Abfragen gegenüber YottaDB überführt werden können.

Damit bildet die aktuelle Grundimplementierung eine stabile Basis, auf der sowohl funktionale Erweiterungen (z. B. Wildcards, Filter, Slices) als auch qualitative Verbesserungen (z. B. differenzierte Fehlermeldungen oder optimierte Auswertungspfade) aufbauen können. Konkrete Erweiterungsszenarien werden im Ausblick (Abschnitt 8) diskutiert.

5 Implementierung

Die Implementierung ist in klar abgegrenzte Module unterteilt, die jeweils einen definierten Aspekt der Verarbeitung übernehmen. Diese Struktur erleichtert nicht nur die Orientierung im Quelltext, sondern unterstützt auch Wartbarkeit und Erweiterbarkeit. Jedes Modul ist dabei einer Schicht des in Kapitel 4 vorgestellten Architekturmodells zugeordnet.

5.1 Code-Struktur

Die Implementierung des Werkzeugs gliedert sich in mehrere klar abgegrenzte Module, die jeweils eine zentrale Teilaufgabe übernehmen. Tabelle 5.1 zeigt die wichtigsten Dateien des Projekts sowie ihre Hauptaufgaben.

Datei	**Funktion**
`scanner.l`	Scanner-Definition (Flex) für die Zerlegung der Eingabe in Tokens.
`parser.y`	Parser-Regeln (Bison), erzeugt den abstrakten Syntaxbaum (AST).
`ast.h/.c`	Definition und Verwaltung der AST-Struktur; enthält Allokation und Pretty-Printer.
`query_yottadb.h/.c`	Übersetzung des AST in YottaDB-Referenzen; Schnittstelle zur Bridge.
`JPBRIDGE.m`	M-Routine im Container, liefert Marker (`JP:D`, `JP:V`) zur Kommunikation mit YottaDB.
`main.c`	Steuerung des CLI-Programms, Verarbeitung der Optionen und Ausführung der Abfragen.
`build.sh`	Skript für Out-of-Source-Build (`build/`, `bin/`).
`run_ydb.sh`	Start und Verwaltung des YottaDB-Docker-Containers.
`Makefile`	Alternative für In-Place-Build, integriert Flex/Bison und GCC.
`README.md`	Projektbeschreibung und kurze Beispiele zur Nutzung.

Tabelle 5.1: Dateien und Hauptaufgaben der Implementierung

Die folgenden Abschnitte vertiefen die Struktur anhand ausgewählter Beispiele aus der Implementierung.

5.2 Scanner

Die Tokenisierung erfolgt durch einen mit Flex generierten Scanner. Er übernimmt die *lexikalische Analyse*, d. h. er zerlegt den Eingabestring in eine Folge von eindeutig typisierten Symbolen (*Tokens*), die anschließend vom Parser weiterverarbeitet werden. Damit wird aus einer unstrukturierten Zeichenkette eine formale Eingabe, die der Grammatik entspricht [5].

Aufbau der Datei

Die Definitionen sind in der Datei `scanner.l` hinterlegt und gliedern sich in drei Teile:

1. **Deklarationen:** Einbindung von Headern und Definition von Hilfsfunktionen, insbesondere zur Übergabe von Attributwerten (`yylval`) an den Parser.
2. **Regeln:** Reguläre Ausdrücke, die konkrete Lexeme erkennen (z. B. `$`, `.name`, `[0]`) und diese in Tokens umwandeln. Bei Bedarf werden Zusatzinformationen wie Zahlenwerte oder Stringinhalte an den Parser weitergereicht.
3. **C-Code:** Hilfsroutinen für Sonderfälle, etwa die Behandlung von Escape-Sequenzen in Strings oder die Konvertierung numerischer Werte.

Eingaben und Ausgaben

Der Scanner arbeitet mit den folgenden Ein- und Ausgaben:

- **Eingabe:** Eine Zeichenkette mit einem JSONPath-Ausdruck, beispielsweise `$.store.book[0].title`.
- **Ausgabe:** Eine Tokenfolge für den Parser, z. B.:
  `DOLLAR`, `DOT`, `NAME("store")`, `DOT`, `NAME("book")`, `LBRACKET`, `NUMBER(0)`, `RBRACKET`, `DOT`, `NAME("title")`.

Damit bildet der Scanner die Grundlage für die anschließende syntaktische Analyse: Er abstrahiert von der Zeichenebene und stellt dem Parser eine klar strukturierte Tokenfolge zur Verfügung.

Unterstützte Tokens

Im aktuell implementierten Teilumfang unterscheidet der Scanner folgende Tokens:

Token	Bedeutung / Beispiel
`DOLLAR`	Wurzelsymbol `$`
`DOT`	Punkt-Operator `.`
`LBRACKET` / `RBRACKET`	Klammern `[ ]`
`NUMBER`	Ganzzahl-Literal (Indexzugriff), z. B. `0`
`NAME`	Unquoted Bezeichner, z. B. `book`
`STRING`	Doppelt-quotierter String, z. B. ¨`title`¨

Tabelle 5.2: Vom Scanner genutzte Tokens im aktuellen Teilumfang

Darüber hinaus sind im Scanner bereits weitere Tokenarten definiert (z. B. `COLON` für Slice-Syntax oder `COMMA` für Unions), die zwar aktuell nicht genutzt werden, aber die spätere Erweiterung der Grammatik erleichtern.

Aufgaben im Gesamtprozess

Der Scanner erfüllt zwei Hauptaufgaben:

- **Lexikalische Analyse:** Er erkennt gültige Lexeme und wandelt sie in Tokens um, sodass der Parser die formale Struktur erkennen kann.
- **Fehlerabgrenzung:** Er weist ungültige Zeichenfolgen frühzeitig ab und liefert so die Grundlage für präzise Fehlermeldungen.

Generierung mit Flex

Die Datei `scanner.l` wird mit Flex in C-Code übersetzt. Der Befehl

```
flex -o lex.yy.c scanner.l
```

erzeugt die Implementierung des Scanners in der Datei `lex.yy.c`, die anschließend mit dem restlichen Projekt kompiliert und gelinkt wird. Dieser Schritt ist im Build-Prozess über `Makefile` bzw. `build.sh` automatisiert, sodass der Scanner bei Änderungen an der `.l`-Datei stets neu generiert wird.

Damit bildet der Scanner die erste Stufe der Verarbeitungspipeline (vgl. Abbildung 4.1). Er entscheidet, welche Eingaben überhaupt weiter analysiert werden können, und legt damit die Basis für eine deterministische Übersetzung nach YottaDB.

5.3 Parser

Der Parser wird mit Bison [4] generiert und verarbeitet Ausdrücke des klar abgegrenzten Teilumfangs. Unterstützt sind Pfadsegmente in Punkt-Notation (`.name`), String-Keys in Klammernotation (`["name"]`) und numerische Indizes (`[0]`). Die Grammatik ist so konstruiert, dass ausschließlich wohlgeformte Sequenzen akzeptiert werden; nicht vorgesehene Konstrukte werden regelbasiert abgewiesen und führen zu einer eindeutigen Fehlermeldung (`EX_PARSE`).

Ziel und Einordnung

Der Parser übernimmt die *syntaktische Analyse* der vom Scanner gelieferten Tokenfolge und überführt sie in eine strukturierte, interne Repräsentation (AST). Er bildet damit die Brücke zwischen der lexikalischen Ebene und der semantischen Ebene.

Aufbau der Datei

Die Bison-Quelldatei `parser.y` gliedert sich in drei Bereiche:

1. **Deklarationsteil:** `%union` für Attributwerte (z. B. `int` für Indizes, `char*` für Feldnamen), `%token`-Definitionen für Terminals (z. B. `DOLLAR`, `DOT`, `LBRACKET`, `RBRACKET`, `NAME`, `STRING`, `NUMBER`) sowie optionale Direktiven wie `%define api.pure` (reentrant) und `%locations` (Positionsinformationen).
2. **Grammatikregeln:** Produktionsregeln für Startsymbol und Nichtterminale; hier werden auch die AST-Knoten aufgebaut (z. B. `make_root()`, `append_field()`, `append_index()`).
3. **C-Code:** Hilfsfunktionen (z. B. `yyerror` für Fehlermeldungen, Allokations-/Freigaberoutinen für AST-Knoten).

Ein- und Ausgaben

Als Eingabe erhält der Parser die vom Scanner gelieferte Tokenfolge, z. B. `DOLLAR`, `DOT`, `NAME(ßtore")`, `DOT`, `NAME("book")`, `LBRACKET`, `NUMBER(0)`, `RBRACKET`, `DOT`, `NAME("title")`. Als Ausgabe erzeugt er einen AST-Wurzelknoten, der diese Sequenz von Pfadsegmenten in hierarchischer Form abbildet und anschließend vom Übersetzer weiterverarbeitet wird.

```
/* Startsymbol: kompletter JSONPath */
jsonpath:
    DOLLAR path    { ASTNode *dollar = make_node(...);
                     *result = make_node(...);  }
  | /* leer */     { yyerror(...); YYERROR; }
  ;
```

Listing 5.1: Startregel für JSONPath im Parser

Bei jeder Reduktion eines Segments wird ein *AST-Knoten* erzeugt und an den laufenden Pfad angehängt. Der Wurzelanker `$` wird als eigener Startknoten modelliert, jedoch nicht in ein Subskript materialisiert. Damit ist der AST eine Liste von Segmentknoten, die das spätere Mapping deterministisch macht. Im Fehlerfall wird ein definierter Rückgabecode (`EX_PARSE`) und eine präzise Fehlermeldung ausgegeben.

Fehlermeldungen

Fehler entstehen, wenn

1. die Reihenfolge oder Form von Segmenten syntaktisch ungültig ist (z. B. fehlende schließende Klammer), oder
2. nicht unterstützte Konstrukte erkannt werden (z. B. `..`, `[*]`, `[a:b]`).

Der Parser erzeugt in diesen Fällen eine spezifische Meldung und beendet die Verarbeitung mit `EX_PARSE`. Optional kann über `%error-verbose` eine erweiterte Fehlerspur aktiviert werden. Positionsinformationen (`%locations`) erlauben die Ausgabe „Zeile/Spalte“.

Unterstützter Teilumfang

Der Prototyp deckt einen bewusst eingeschränkten Teil der JSONPath-Syntax ab. Die folgende Tabelle zeigt die derzeit unterstützten Konstrukte mit Beispielen.

Unterstützt	**Beispiele**
Wurzel	`$`
Punkt-Notation	`$.bicycle.color`
String-Keys	`$["bicycle"]["price"]`
Ganzzahl-Indizes	`$.book[0].title`

Tabelle 5.3: Unterstützte JSONPath-Konstrukte

Nicht unterstützte Konstrukte

Über den Kernumfang hinausgehende Sprachmerkmale werden bewusst ausgeschlossen. Sie führen im Parser zu einer eindeutigen Fehlermeldung.

Nicht unterstützt	Fehlermeldung (Exitcode `EX_PARSE`)
Rekursiver Deszent `..`	`rekursiver Deszent nicht unterstützt`
Wildcards `.*`, `[*]`	`Wildcards nicht unterstützt`
Slices `[a:b]`, `[a:b:c]`	`Array-Slices nicht unterstützt`
Filter `?()`	`Filter nicht unterstützt`
Unions `[a,b]`	`Unions nicht unterstützt`
Negative Indizes	`negative Indizes nicht erlaubt`

Tabelle 5.4: Aktuell nicht unterstützte JSONPath-Konstrukte und Fehlermeldungen

EBNF-Grammatik

Der in `parser.y` implementierte Teilumfang lässt sich in EBNF-ähnlicher Notation formal ausdrücken. Listing 5.2 und 5.3 zeigen einen Auszug der Grammatikregeln, ergänzt um Kommentare, welche nicht unterstützten Konstrukte explizit abgewiesen werden.

```
jsonpath   ::= "$" path
             | /* Fehler: leerer Ausdruck */

path       ::= member path
             | subscript path
             | /* Epsilon */
             | /* Fehler: ".." (rekursiver Deszent) nicht
   unterstützt */

member     ::= "." NAME
             | /* Fehler: ".*" (Wildcard im Objektzugriff) nicht
   unterstützt */

subscript  ::= "[" sub_expr "]"

sub_expr   ::= array_index
             | member_name
             | /* Fehler: "[*]" (Wildcard), "?()" (Filter),
                        ":" (Slices), "," (Union) nicht
   unterstützt */
```

Listing 5.2: EBNF-Auszug JSONPath-Grammatik

```
array_index::= INT
             | /* Fehler: NUMBER (nicht-ganzzahlig/negativ) nicht
   erlaubt */

member_name::= STRING
```

Listing 5.3: EBNF-Auszug JSONPath-Grammatik

Die Angabe verdeutlicht, dass die Grammatik bewusst minimal gehalten ist und fehlerhafte oder nicht implementierte Ausdrucksformen direkt auf Parser-Ebene erkannt werden. Dadurch wird sichergestellt, dass nur deterministisch übersetzbare Ausdrücke an den AST weitergegeben werden.

Beispielhafte Ableitung und AST

Für `$.store.book[0].title` ergibt sich skizzenhaft die folgende Ableitung: Ausgehend vom Startsymbol `jsonpath` reduziert der Parser die vom Scanner gelieferten Token `DOLLAR`, `DOT`, `NAME("store")`, `DOT`, `NAME("book")`, `LBRACKET`, `NUMBER(0)`, `RBRACKET`, `DOT`, `NAME("title")` Schritt für Schritt zu einer hierarchischen Struktur aus `path`-, `member`- und `subscript`-Knoten.

Die Abbildung 5.1 zeigt den daraus entstehenden AST in vereinfachter Form:

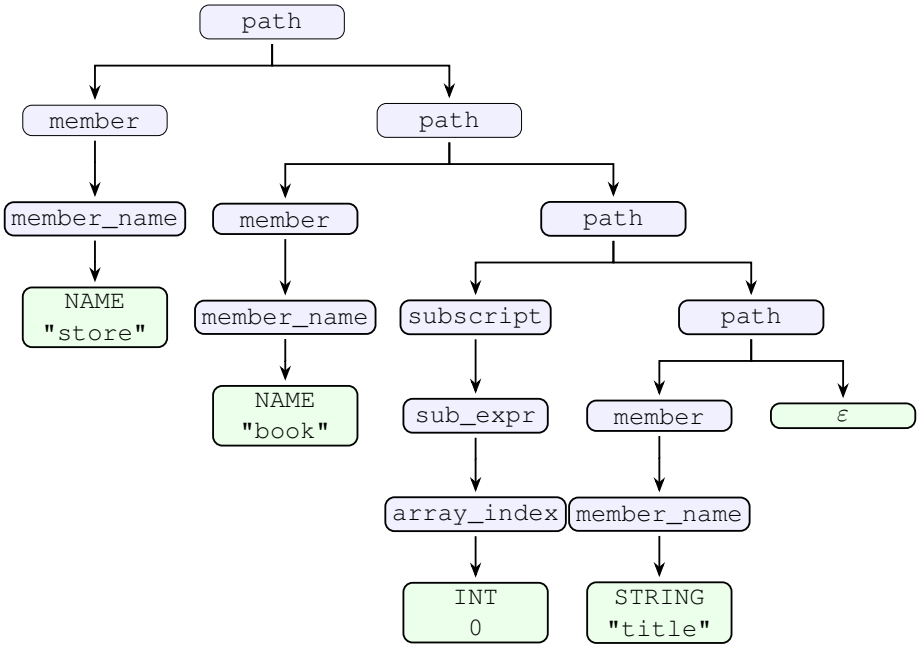

Abbildung 5.1: Beispiel abgeleiteter AST

Dabei wird jeweils ein neuer AST-Knoten erzeugt und mit dem bestehenden Teilbaum verknüpft. Das Resultat ist ein Baum, in dem jedes Pfadsegment („store“, „book“, Index 0, „title“) durch ein eigenes Teilobjekt repräsentiert ist. Der Wurzelanker `$` dient als Startknoten, wird aber nicht als Subskript materialisiert.

Generierung mit Bison

Die Datei `parser.y` wird mit Bison in C-Code übersetzt:

```
bison -d -o parser.tab.c parser.y
```

Dabei erzeugt Bison die Parser-Implementierung (`parser.tab.c`) sowie die zugehörige Headerdatei (`parser.tab.h`) mit Token- und `%union`-Definitionen. Der Build ist über `Makefile` bzw. `build.sh` automatisiert, sodass der Parser bei Änderungen an `parser.y` neu generiert wird.

5.4 AST-Implementierung und Pretty-Printer

Der abstrakte Syntaxbaum (AST) stellt die Struktur eines JSONPath-Ausdrucks in Baumform dar. Er wird vom Parser erzeugt und anschließend für die Übersetzung in eine YottaDB-Referenz genutzt.

Struktur der Knoten

Die Knoten sind in `ast.h` als `struct ASTNode` definiert.

```
typedef struct ASTNode {
    ASTNodeType type;
    struct ASTNode *left;
    struct ASTNode *right;
    char *value;
} ASTNode;
```

Listing 5.4: struct ASTNode

In jedem Knoten sind drei Felder hinterlegt: `type` kennzeichnet den Knotentyp als `enum ASTNodeType` (etwa `AST_IDENTIFIER` für Feldnamen oder `AST_LITERAL` für Indizes). Über `left` und `right` verweisen die Knoten auf ihre Kindknoten; gemeinsam bilden diese Zeiger die Verkettung bzw. den Baum der Pfadsegmente. Das Feld `value` enthält schließlich den Stringwert des Knotens, also z. B. den konkreten Feldnamen oder ein Indexliteral.

Knotentypen

Die Knotentypen decken den aktuellen Teilumfang ab, sind aber bereits auf zukünftige Erweiterungen vorbereitet (vgl. Tabelle 5.5).

Typ	Bedeutung / Verwendung
`AST_PATH`	Verkettung von Segmenten zu einem vollständigen Pfad.
`AST_IDENTIFIER`	Feldzugriff per Name (`.name` oder `["name"]`).
`AST_LITERAL`	Ganzzahl-Literal für Array-Indizes (`[0]` → Wert `"0"`).
`AST_FILTER`, `AST_UNION`, `AST_STAR`, `AST_SLICE`, ...	Für spätere JSONPath-Konstrukte reserviert.

Tabelle 5.5: AST-Knotentypen im Projekt

AST-Erzeugung

Der Parser ruft beim Reduzieren von Grammatikregeln die Funktion `make_node()` auf, um für jedes Segment einen Knoten zu erzeugen. Diese werden über die Felder `left`/`right` zu einer Kette verbunden. Das Ergebnis ist ein Baum, der die hierarchische Struktur des Pfades widerspiegelt und die Grundlage für die Übersetzung nach YottaDB bildet.

```
1  ASTNode *make_node(ASTNodeType type, ASTNode *left, ASTNode
       *right, const char *value) {
2      ASTNode *node = malloc(sizeof(ASTNode));
3      if (!node) return NULL;
4
5      node->type  = type;
6      node->left  = left;
7      node->right = right;
8
9      if (value) {
10         node->value = strdup(value); // eigene Kopie anlegen
11     } else {
12         node->value = NULL;
13     }
14     return node;}
```

Listing 5.5: Knotenerzeugung

Die Funktion allokiert einen neuen `ASTNode` auf dem Heap und initialisiert ihn mit Typ, optionalem Kindknoten (`left`, `right`) und einem Wert. Für Strings wird eine Kopie mit `strdup()` erzeugt, sodass der Knoten unabhängig vom ursprünglichen Eingabespeicher existiert. Damit wird sichergestellt, dass der AST stabil im Speicher steht, auch wenn der Parser intern Puffer wiederverwendet.

Hilfsfunktionen

Neben der Kernfunktion existieren Hilfsfunktionen, die die Arbeit mit dem AST erleichtern:

- `free_ast(...)`: rekursive Freigabe aller Knoten und ihrer Werte.
- `print_ast(...)`: rekursive Ausgabe zur Visualisierung des Baums.

Pretty-Printer Die Funktion `print_ast()` dient als Debug- und Analysewerkzeug. Sie durchläuft den Baum rekursiv und gibt die Knoten mit Einrückungen aus, sodass die hierarchische Struktur sichtbar wird.

Ein Beispielaufruf mit der Option `-a` im CLI erzeugt für `$.book[0].title` folgende Ausgabe:

```
PATH
  IDENTIFIER: $
  PATH
    IDENTIFIER: book
    PATH
      LITERAL: 0
      IDENTIFIER: title
```

Listing 5.6: struct ASTNode

Im regulären Betrieb wird der Pretty-Printer nicht benötigt, er ist jedoch hilfreich, um zu prüfen, ob Scanner und Parser den Eingabestring korrekt in einen AST überführt haben.

5.5 YDB-Translator

Der Translator bildet die Brücke zwischen dem AST und der konkreten YottaDB-Referenzsyntax. Das Modul `query_yottadb.h/.c` erzeugt aus einem gegebenen AST eine Zeichenkette, die eine gültige Referenz darstellt.

5.5.1 Erstellen der YDB-Referenz

Die zentrale Schnittstelle ist die Funktion `build_ydb_ref`. Sie überprüft ihre Eingabeparameter, setzt das Flag `*unsupported` auf null und schreibt anschließend das Präfix mit Globalnamen in den vom Aufrufer bereitgestellten Ausgabepuffer. Danach ruft sie den rekursiven Traversierer `walk_ast` auf, der die einzelnen Subskripte aus dem AST generiert. Zum Schluss sorgt `finalize_ref` für eine korrekt geschlossene Klammer.

```
1  int build_ydb_ref(const ASTNode *ast, const char *root_global,
2                    char *out, size_t outsz, int *unsupported)
3  {
4      if (unsupported)
5          *unsupported = 0;
6      if (!out || outsz == 0 || !ast || !root_global ||
       !*root_global)
7          return -1;
8
9      // Ausgabepuffer leeren
10     out[0] = '\0';
11
12     // Präfix schreiben: ^<root>(
13     buf_cat(out, outsz, "^");
14     buf_cat(out, outsz, root_global);
15     buf_cat(out, outsz, "(");
16
17     // AST rekursiv traversieren
18     int need_comma = 0;
19     walk_ast(ast, out, outsz, &need_comma, unsupported);
20
21     // Referenz schließen (Klammer setzen)
22     finalize_ref(out, outsz);
23
24     return 0;
25 }
```

Listing 5.7: Vollständige Funktion `build_ydb_ref`

Die Funktion gibt `0` bei Erfolg und `-1` bei fehlerhaften Parametern zurück. Der resultierende Referenzstring liegt direkt im vom Aufrufer bereitgestellten Puffer `out`.

Die Details der Übersetzung liegen nicht in `build_ydb_ref` selbst, sondern werden an `walk_ast` delegiert.

5.5.2 Traversierung

Die zentrale Arbeit übernimmt die Funktion `walk_ast`. Sie wird rekursiv aufgerufen und entscheidet für jeden Knoten, welches Subskript in den Ausgabepuffer geschrieben wird. Parameter sind der aktuelle Knoten, ein Zielpuffer mit Größe sowie zwei Flags: `need_comma` zur Steuerung der Kommasetzung und `unsupported`, um nicht implementierte Knotentypen zu markieren.

```
1  static void walk_ast(const ASTNode *n, char *out, size_t outsz,
       int *need_comma, int *unsupported)
2  {
3      if (!n)
4          return;
5
6      switch (n->type)
7      {
8      case AST_PATH:
9          walk_ast(n->left, out, outsz, need_comma, unsupported);
10         walk_ast(n->right, out, outsz, need_comma, unsupported);
11         return;
12
13     case AST_IDENTIFIER:
14         if (n->value && strcmp(n->value, "$") == 0)
15             return; /* Wurzel */
16         if (*need_comma)
17             buf_cat(out, outsz, ",");
18         {
19             char esc[2048];
20             m_escape(n->value ? n->value : "", esc, sizeof esc);
21             buf_cat(out, outsz, "\"");
22             buf_cat(out, outsz, esc);
23             buf_cat(out, outsz, "\"");
24         }
25         *need_comma = 1;
26         return;
```

Listing 5.8: Traversierer `walk_ast` (aus `query_yottadb.c`)

```
    case AST_LITERAL:
        if (*need_comma)
            buf_cat(out, outsz, ",");
        {
            long idx = strtol(n->value ? n->value : "0", NULL, 10);
            char num[64];
            snprintf(num, sizeof num, "%ld", idx + 1); /*
    0-basiert -> 1-basiert */
            buf_cat(out, outsz, num);
        }
        *need_comma = 1;
        return;

    default:
        if (unsupported)
            *unsupported = 1;
        return;
    }
}
```

Listing 5.9: Traversierer `walk_ast` (aus `query_yottadb.c`)

Die Funktion arbeitet wie folgt:

- **AST_PATH** ruft sich zuerst auf dem linken und dann auf dem rechten Teilbaum auf. So bleibt die Reihenfolge der Pfadsegmente erhalten.
- **AST_IDENTIFIER** behandelt Feldnamen. Der Sonderfall `$` als Wurzel wird ignoriert. Ansonsten wird – ggf. mit vorangestelltem Komma – ein String in Anführungszeichen ausgegeben, wobei `m_escape` innere Anführungszeichen verdoppelt.
- **AST_LITERAL** verarbeitet Array-Indizes. Das Literal wird mit `strtol` in eine Ganzzahl gewandelt, um eins erhöht (wegen der Umstellung von 0-basiert auf 1-basiert) und als Zahl ausgegeben.
- **Default:** unbekannte Knotentypen setzen das Flag `*unsupported` und führen so zu einem Abbruchsignal.

Die Kommasetzung steuert das Flag `need_comma`: beim ersten Subskript wird kein Komma gesetzt, ab dem zweiten immer.

5.5.3 Hilfsfunktionen

Eine Reihe kleiner Hilfsfunktionen unterstützt den Traversierer beim sicheren Aufbau der Referenz. Sie kapseln wiederkehrende Aufgaben wie String-Anfügung, Quoting oder das korrekte Schließen der Klammer.

Sicheres Anhängen Das Anhängen neuer Zeichenketten an den Ausgabepuffer übernimmt `buf_cat`. Die Funktion prüft die aktuelle Länge des Puffers, schneidet die Eingabe bei Bedarf ab und stellt sicher, dass die Zeichenkette immer korrekt mit `NUL` terminiert bleibt.

```
static void buf_cat(char *buf, size_t size, const char *s)
{
    if (!buf || !s || size == 0) return;
    size_t bl = strlen(buf), sl = strlen(s);
    if (bl >= size - 1) return;
    if (sl > size - bl - 1) sl = size - bl - 1;
    memcpy(buf + bl, s, sl);
    buf[bl + sl] = '\0';
}
```

Listing 5.10: `buf_cat` (aus `query_yottadb.c`)

Escaping von Zeichenketten Schlüssel in JSONPath können Anführungszeichen enthalten, die in M/MUMPS durch Verdopplung (`"` → `""`) dargestellt werden. Die Funktion `m_escape` übernimmt dieses Escaping und kopiert den Eingabestring entsprechend angepasst in den Zielpuffer.

```
static void m_escape(const char *in, char *out, size_t outsz)
{
    if (!out || outsz == 0) return;
    size_t j = 0;
    if (!in) { out[0] = '\0'; return; }
    for (size_t i = 0; in[i] && j + 2 < outsz; ++i) {
        if (in[i] == '"') { out[j++] = '"'; out[j++] = '"'; }
        else { out[j++] = in[i]; }
    }
    out[j] = '\0';
}
```

Listing 5.11: `m_escape` (aus `query_yottadb.c`)

Abschluss der Referenz Nachdem alle Subskripte emittiert wurden, sorgt `finalize_ref` für einen sauberen Abschluss der Referenz. Endet der Puffer auf einem überzähligen Komma, wird dieses entfernt und eine schließende Klammer angefügt. Fehlt die Klammer, wird sie ebenfalls ergänzt.

```
static void finalize_ref(char *ref, size_t refsz)
{
    if (!ref || !*ref) return;
    size_t len = strlen(ref);
    if (len == 0) return;
    if (ref[len - 1] == ',') { ref[len - 1] = '\0'; buf_cat(ref,
   refsz, ")"); }
    else if (ref[len - 1] != ')') { buf_cat(ref, refsz, ")"); }
}
```

Listing 5.12: `finalize_ref` (aus `query_yottadb.c`)

Beispiel

Für den Ausdruck `$.book[0].title` schreibt `build_ydb_ref` zunächst das Präfix `^store(` in den Ausgabepuffer. `walk_ast` emittiert anschließend `"book"`, den um $+1$ verschobenen Index `1` (aus `0`) sowie `"title"`, jeweils mit korrekter Kommasetzung über `need_comma`. `finalize_ref` fügt die schließende Klammer an. Das Ergebnis lautet:

store("book",1,"title")

Tritt ein nicht unterstützter Knotentyp auf, setzt `walk_ast *unsupported = 1`, sodass der Aufrufer den Fehlerfall erkennen kann.

Fehlerfälle und Cleanup

Fehler werden auf zwei Wegen signalisiert: Zum einen prüft `build_ydb_ref` seine Eingabeparameter und liefert bei ungültigen Argumenten einen negativen Rückgabewert (`-1`). Zum anderen setzt `walk_ast` das Flag `*unsupported`, sobald ein Knotentyp angetroffen wird, der nicht implementiert ist. In diesem Fall wird die Traversierung zwar beendet, der Aufrufer erkennt aber anhand des Flags, dass die Referenz nicht verwendbar ist.

Auch das Parsen von Literalen erfolgt defensiv über `strtol`. Falls die Eingabe nicht in eine Ganzzahl übersetzt werden kann, wird als Fallback der Wert 0 verwendet und anschließend wie gewohnt um eins erhöht. So bleibt die Referenzbildung robust, auch wenn die Eingabe nicht exakt der Spezifikation entspricht.

Da der Ausgabepuffer vom Aufrufer bereitgestellt wird, entfällt eine interne Allokation oder Freigabe. Der Translator schreibt direkt in den Puffer; Cleanup ist somit Aufgabe des Aufrufers nur insofern, als der Puffer groß genug dimensioniert sein muss.

Komplexität und Robustheit

Die Laufzeit des Translators ist linear in der Anzahl der AST-Knoten: Jeder Knoten wird genau einmal besucht und in den Ausgabepuffer übersetzt. Die Pufferoperationen in `buf_cat` hängen Zeichenketten mit Längenprüfung an und garantieren `NUL`-Terminierung. Dynamische Speichervergrößerungen finden nicht statt; die Effizienz hängt daher von einer ausreichenden Puffergröße ab, die der Aufrufer vorgibt.

Der Translator arbeitet rein syntaktisch: Er verändert den Datenbankzustand nicht, sondern liefert ausschließlich Strings, die eine gültige YottaDB-Referenz darstellen. Durch die Kombination aus defensivem Parsing, sicherem Puffer-Handling und explizitem Unsupported-Flag ist die Implementierung robust gegen fehlerhafte Eingaben und stellt sicher, dass keine undefinierten Zustände entstehen.

5.6 Build-Prozess

Die Erstellung des Executables erfolgt in einer klar definierten Abfolge:

1. **Scanner-Generierung:** Flex übersetzt `scanner.l` in `lex.yy.c`.
2. **Parser-Generierung:** Bison erzeugt aus `parser.y` die Dateien `parser.tab.c` und `parser.tab.h`.
3. **Kompilation:** Die Quell- und Headerdateien (`ast.c/.h`, `query_yottadb.c/.h`, `main.c`) sowie die generierten Scanner-/Parser-Dateien werden kompiliert.
4. **Linking:** Alle Objektdateien werden mit der Flex-Laufzeitbibliothek (`-lfl`) zu einem Executable zusammengefügt.
5. **Ergebnis:** Das fertige Programm liegt in `bin/jp2ydb`.

Das Skript `build.sh` automatisiert diesen Ablauf und kapselt die Kommandos für Flex, Bison und GCC. Die Steuerung basiert auf Makefiles [6]. Sämtliche Artefakte werden in `build/` abgelegt, wodurch Quellcode und generierte Dateien klar getrennt bleiben.

Standardmäßig wird ein optimierter Release-Build erstellt. Mit dem Argument

`debug` werden zusätzliche Debug-Symbole und Compiler-Warnungen (z. B. `-Wall -Wextra`) aktiviert. Der Aufruf `clean` entfernt alle Artefakte, wodurch ein reproduzierbarer Neuaufbau jederzeit möglich ist.

> **Hinweis:** Unter macOS ist häufig `-ll` anstelle von `-lfl` erforderlich. Unter Debian/Ubuntu wird das Paket `libfl-dev` benötigt.

Artefakte und Abhängigkeiten

Die durch den Build erzeugten Dateien werden strikt von den manuell gepflegten Quellen getrennt, um eine saubere Projektstruktur zu gewährleisten. Zusätzlich sind bestimmte Werkzeuge und eine Laufzeitumgebung (Docker mit YottaDB) erforderlich, damit das CLI lauffähig ist.

- **Artefakte:** `build/` enthält ausschließlich generierte Parser-/Scanner-Dateien und Objektdateien; `bin/jp2ydb` ist das fertige Executable.
- **Abhängigkeiten:** GCC/Clang, Flex, Bison; zur Laufzeit zusätzlich Docker mit YottaDB-Image. Fehlt die Bridge, wird `JPBRIDGE.m` automatisch in den Container kopiert und kompiliert.

  > **Hinweis:** Der Name des Docker-Containers wird über die Umgebungsvariable `YDB_CONTAINER` bestimmt. Ist diese nicht gesetzt, fällt das Programm standardmäßig auf den Container „ydb“ zurück.

5.7 Zugriffsschicht und CLI

Die Zugriffsschicht verbindet die vom Translator erzeugte YottaDB-Referenz mit der Ausführung im Container. Sie besteht aus einer M-Routine im Container (`JPBRIDGE.m`), die für eine übergebene Referenz deterministische Marker emittiert, und aus Host-seitigen C-Funktionen, die diese Marker auswerten und über die CLI zurückgeben.

Ablauf im Hostprogramm

Der Ablauf im Host beginnt mit der Referenzbildung aus dem AST (vgl. Abschnitt „YDB-Translator“, Abschnitt 5.5). Die eigentliche Anfrage an YottaDB führt `ydb_get` aus, das intern die Bridge im Container nutzt (vgl. die Abschnitte „Bereitstellung der Bridge“, Abschnitt 5.7, und „Bridge-Routine“, Abschnitt 5.7). Vor der Ausführung

wird die Verfügbarkeit der Bridge geprüft und bei Bedarf hergestellt (vgl. „Bereitstellung der Bridge“, Abschnitt 5.7). Anschließend wird die Routine aufgerufen; die ausgegebenen Marker werden eingelesen und interpretiert (vgl. „Marker-Auswertung im Host“, Abschnitt 5.7). Ausgabeformate, Betriebsmodi und Diagnoseverhalten sind in der CLI zusammengefasst (vgl. „CLI-Integration und Optionen“, Abschnitt 5.7). Ein Überblick des Gesamtablaufs findet sich im Sequenzdiagramm (vgl. Abbildung 5.2).

Bereitstellung der Bridge

Bevor eine Anfrage ausgeführt wird, stellt der Host sicher, dass die Routine `JPBRIDGE.m` im Container vorhanden und aufrufbar ist. Diese Prüfung und ggf. Provisionierung erfolgt im Rahmen des Aufrufs von `ydb_get` und umfasst das Bereitstellen der Routine im Container sowie eine kurze Funktionsprobe. Ist die Bridge bereits vorhanden, wird der Schritt übersprungen.

Bridge-Routine

Die Routine `JPBRIDGE.m` im Container implementiert die Marker-Emission. Sie liest die Referenz aus dem Kommandozeilenargument (`$ZCMD`), prüft den Knotenstatus mit `$DATA` und gibt definierte Marker auf `stdout` aus:

```
JPBRIDGE ; generic bridge
 NEW ref SET ref=$ZCMD
 NEW d SET d=$DATA(@ref) WRITE "JP:D:",d,!
 NEW v SET v=$GET(@ref) WRITE "JP:V:",v,!
 QUIT
```

Listing 5.13: Die Routine `JPBRIDGE.m`

Damit wird für jede Referenz zunächst der Status nach `$DATA` zurückgemeldet; `JP:V:` enthält den zugehörigen Skalarwert. In der Auswertung auf dem Host wird `JP:V:` nur dann als Nutzwert übernommen, wenn `JP:D:` den Vorhandenseinstatus eines Werts anzeigt.

Marker-Auswertung im Host

Die Marker werden Host-seitig zeilenweise eingelesen und deterministisch interpretiert. Der Datenstatus $d =$ `JP:D:` unterscheidet vier Fälle: Bei $d = 0$ existiert der Pfad nicht und ein ggf. ausgegebenes `JP:V:` bleibt ohne Wirkung; bei $d = 1$ liegt ein Skalarwert vor, `JP:V:` wird als Ergebnis übernommen; bei $d = 10$ existieren nur Kindknoten, die aktuelle Implementierung liefert hierfür einen Platzhalter; bei

$d = 11$ sind Wert und Kinder vorhanden, der Wert aus `JP:V:` wird übernommen, Kindschlüssel werden (noch) nicht ausgegeben.

CLI-Integration und Optionen

Die CLI bindet die Zugriffsschicht in ein konsistentes Interface ein: Nutzwerte werden auf `stdout` ausgegeben, Fehlermeldungen und Diagnosen auf `stderr`. Die Modi `-a` (AST), `-r` (nur Referenz) und `-q` (nur Wert) trennen Darstellung und Abfrage. Die Exit-Codes signalisieren den Pfad der Fehlerursache (`EX_OK`=0, `EX_PARSE`=65, `EX_NOTFOUND`=66, `EX_YDB`=67). Optionales Debug-Logging lässt sich über ein Umgebungsflag aktivieren und protokolliert die relevanten Shell-Kommandos sowie die eingehenden Marker.

Sequenzdiagramm

Das folgende Sequenzdiagramm stellt den Ablauf einer Abfrage von der Kommandozeilenoberfläche bis zur Auswertung der Marker dar. Es zeigt die beteiligten Komponenten – *Hostprogramm (CLI)*, den *Docker-Container*, die darin laufende *YottaDB*-Instanz sowie die *Bridge-Routine* – und die zwischen ihnen ausgetauschten Nachrichten. Dabei wird deutlich, wie aus der im Host erzeugten Referenz ein Aufruf der Routine `JPBRIDGE.m` entsteht, welche Marker (`JP:D:`, `JP:V:`) emittiert und wieder zurück an den Host transportiert werden. Die eigentliche Interpretation dieser Marker erfolgt anschließend im Hostprogramm.

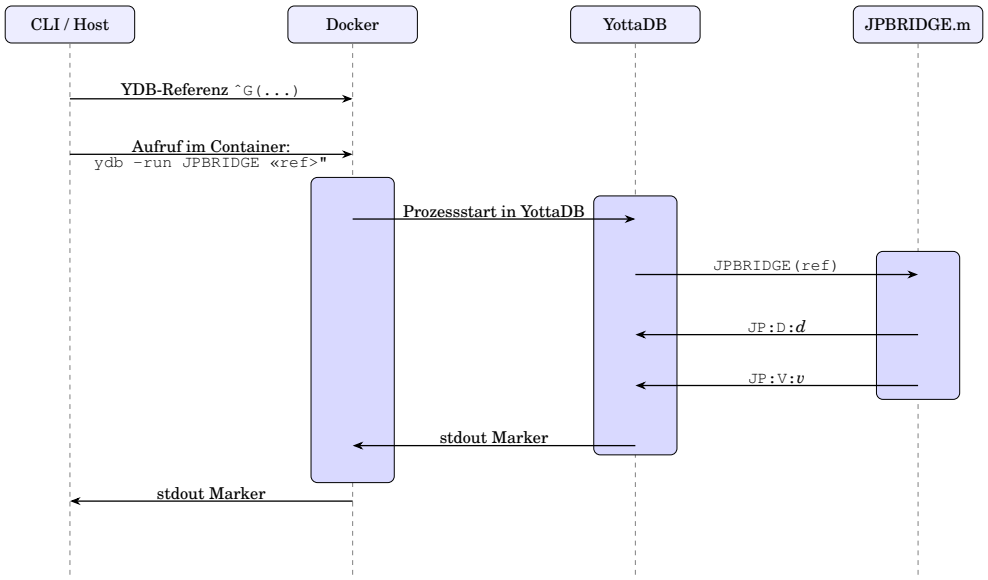

Abbildung 5.2: Ablauf von der Referenzbildung bis zur Marker-Auswertung

6 Inbetriebnahme

Dieses Kapitel dokumentiert die Schritte zur Reproduktion der Entwicklungs- und Laufzeitumgebung. Ziel ist es, die Implementierung auf einem frischen System nachvollziehbar zu kompilieren und in Betrieb zu nehmen.

6.1 Voraussetzungen und Installation

Systemvoraussetzungen

Für die Kompilierung und Ausführung werden folgende Komponenten benötigt:

- Betriebssystem: Linux (getestet unter Ubuntu 22.04), alternativ macOS mit angepasster Linker-Option (`-ll`).
- Compiler und Tools: GCC oder Clang, Flex, Bison.
- YottaDB-Docker-Image (getestet mit `yottadb-debian:1.38`).

Build-Prozess

Empfohlen wird der Build über das Skript `build.sh`, da hierbei ein Out-of-Source-Build durchgeführt und die Artefakte in separaten Verzeichnissen abgelegt werden:

```
./build.sh            # Release-Build (optimierte Version)
./build.sh debug      # Debug-Build (mit Symbolen)
./build.sh clean      # Aufräumen (löscht build/ und bin/)
# Ergebnis: bin/jp2ydb
```

Alternativ kann das Makefile direkt verwendet werden. In diesem Fall wird das Binary im Projektverzeichnis erzeugt:

```
make clean && make
# Ergebnis: ./jp2ydb
```

Container vorbereiten

Das Hilfsskript `run_ydb.sh` automatisiert den Start des YottaDB-Containers [3]. Es führt dabei folgende Schritte aus:

1. Anlegen eines lokalen Datenverzeichnisses `build/ydb-data/`, das in den Container eingebunden wird und persistente YottaDB-Daten enthält.
2. Entfernen eines evtl. vorhandenen Containers mit demselben Namen (`ydb`, überschreibbar über `YDB_CONTAINER`).
3. Start eines neuen Containers auf Basis des offiziellen YottaDB-Images mit interaktiver YottaDB-Shell.

```
# Container starten (Standardname: ydb)
./run_ydb.sh
```

Hinweis: Der Container bleibt aktiv solange diese Sitzung läuft, und wird beim Beenden durch `halt` oder `Ctrl+D` automatisch entfernt

Container mit Beispieldaten füllen

Für die Demonstration wird ein Dataset im Container angelegt:

```
docker exec -it ydb bash -lc '
source /opt/yottadb/current/ydb_env_set
ydb <<"M"
; bicycle
SET ^store("bicycle","color")="red"
SET ^store("bicycle","price")=399
; book[0]
SET ^store("book",1,"category")="fiction"
SET ^store("book",1,"author")="Herman Melville"
SET ^store("book",1,"title")="Moby Dick"
SET ^store("book",1,"price")=8.99
; book[1]
SET ^store("book",2,"category")="fiction"
SET ^store("book",2,"author")="J. R. R. Tolkien"
SET ^store("book",2,"title")="The Lord of the Rings"
SET ^store("book",2,"price")=22.99
; warehouse
SET ^warehouse("location")="Berlin"
SET ^warehouse("capacity")=1200
HALT
M
'
```

Ausführung

Eine typische Abfrage lautet:

```
./bin/jp2ydb '$.bicycle.color'
```

Dies liefert bei obigem Dataset die erwartete Ausgabe:

```
red
```

6.2 Beispieldaten und Referenzaufrufe

6.2.1 Überblick über das Beispieldataset

Das in Abschnitt 6.1 angelegte Dataset umfasst ein Fahrrad, zwei Bücher sowie ein Lager. Die folgende Übersicht zeigt typische Pfade, ihre YottaDB-Referenz und die erwarteten Werte:

Tabelle 6.1: Beispieldaten: JSONPath, YottaDB-Referenz und erwarteter Wert

JSONPath	YDB-Referenz	Wert
`$.bicycle.color`	`^store("bicycle","color")`	`red`
`$.book[0].title`	`^store("book",1,"title")`	`Moby Dick`
`$.book[0].price`	`^store("book",1,"price")`	`8.99`
`$.capacity`	`^warehouse("capacity")`	`1200`

6.2.2 Abfragen über das CLI

Das CLI unterstützt verschiedene Ausgabeoptionen, die insbesondere für Debugging und Entwicklung nützlich sind:

```
# Nur die übersetzte YottaDB-Referenz anzeigen
./bin/jp2ydb -r '$.bicycle.color'
> YottaDB-Abfrage (Global ^store):
[YDB] Referenz: ^store("bicycle","color")
red

# Nur den AST anzeigen
> Ast-Ausgabe:
PATH
  IDENTIFIER: $
  PATH
    IDENTIFIER: book
    PATH
      LITERAL: 0
      PATH
        IDENTIFIER: title
Moby Dick

# AST, Referenz und Wert gemeinsam ausgeben
./bin/jp2ydb -ar '$.book[0].price'
> Ast-Ausgabe:
PATH
  IDENTIFIER: $
  PATH
    IDENTIFIER: book
    PATH
      LITERAL: 0
      PATH
        IDENTIFIER: price
> YottaDB-Abfrage (Global ^store):
[YDB] Referenz: ^store("book",1,"price")
8.99
```

Hinweis: Die kombinierte Option `-ar` ist besonders hilfreich für die Nachvollziehbarkeit der Übersetzungskette von JSONPath über den AST zur konkreten YottaDB-Abfrage.

6.2.3 Alternative Roots

Standardmäßig wird das Global `^store` als Wurzel für JSONPath-Ausdrücke verwendet. Mit der Option `-g` lässt sich ein alternatives Global setzen, z. B. `^warehouse`.

```
./bin/jp2ydb -g warehouse '$.capacity'
1200
```

6.2.4 Fehlerfälle und Negativtests

Die folgenden Beispiele zeigen das Verhalten bei fehlerhaften oder nicht unterstützten Ausdrücken:

```
$ ./bin/jp2ydb '$.unknown'
Fehler: Pfad nicht gefunden.

$ ./bin/jp2ydb '$..book'
Syntaxfehler: Rekursiver Deszent (..) ist nicht unterstützt
```

6.2.5 Manuelle Tests der Bridge

Um die Funktionsfähigkeit der Bridge und die Anbindung an den Container zu verifizieren, können die folgenden manuellen Tests ausgeführt werden. Dies ist insbesondere für Debugging oder zur Nachvollziehbarkeit nützlich.

Bridge manuell testen/kompilieren (optional):

```
docker cp JPBRIDGE.m ${YDB_CONTAINER:-ydb}:/tmp/
docker exec -i ${YDB_CONTAINER:-ydb} bash -lc '
  source /opt/yottadb/current/ydb_env_set
  export ydb_routines="/tmp $ydb_routines"
  mumps /tmp/JPBRIDGE.m
'
```

> **Hinweis:** Eine manuelle Kompilierung der Bridge ist im Normalbetrieb nicht erforderlich. Der CLI-Adapter prüft beim ersten Zugriff automatisch, ob die Routine im Container verfügbar ist, und übernimmt gegebenenfalls das Kopieren und Kompilieren eigenständig.

Beispiel-Lesezugriff Das CLI gibt bei einer Abfrage ausschließlich den Nutzwert (hier: `red`) aus. Die direkte Ausführung der Bridge liefert zusätzlich die Marker `JP:D` und `JP:V`, welche intern vom Host-Programm interpretiert werden.

```
docker exec -i ${YDB_CONTAINER:-ydb} bash -lc '
  source /opt/yottadb/current/ydb_env_set
  export ydb_routines="/tmp $ydb_routines"
  ydb -run JPBRIDGE "^store(\"bicycle\",\"color\")"
'
```

Die Ausgabe für das in Abschnitt 6.1 angelegte Dataset lautet:

```
JP:D:1
JP:V:red
```

6.2.6 Debugging mit JP_DEBUG

Für die Fehlersuche kann die Umgebungsvariable `JP_DEBUG` auf `1` gesetzt werden. Das CLI protokolliert dann zusätzlich die ausgeführten Shell-Kommandos sowie alle empfangenen Marker auf `stderr`. Dies erleichtert die Nachvollziehbarkeit der Containerinteraktion.

6.2.7 Interaktiver Modus

Wird das Programm ohne Argumente gestartet, wechselt es in einen interaktiven Modus. Hier können mehrere JSONPath-Ausdrücke nacheinander eingegeben werden, ohne das Programm jedes Mal neu starten zu müssen:

```
./bin/jp2ydb -ar
Gib einen JSONPath-Ausdruck ein (oder 'exit'):
> $.book[1].title
> Ast-Ausgabe:
PATH
  IDENTIFIER: $
  PATH
    IDENTIFIER: book
    PATH
      LITERAL: 1
      PATH
        IDENTIFIER: title
> YottaDB-Abfrage (Global ^store):
[YDB] Referenz: ^store("book",2,"title")
The Lord of the Rings
```

6.3 Konfiguration und Reproduzierbarkeit

Die Ausführung des CLI hängt sowohl von Umgebungsvariablen als auch von Kommandozeilenoptionen ab. In diesem Abschnitt wird beschrieben, wie diese Konfiguration erfolgt und wie sich ein reproduzierbarer Zustand herstellen lässt.

Umgebungsvariablen

Zur Steuerung der Laufzeitumgebung können verschiedene Variablen gesetzt werden. Sie dienen insbesondere zur Anpassung an unterschiedliche Container-Namen oder Dateipfade. Nicht gesetzte Variablen werden durch sinnvolle Defaults ersetzt.

- `YDB_CONTAINER` – Name des Docker-Containers (Default: `ydb`).
- `JP_BRIDGE_FILE` – Pfad zur M-Routine der Bridge (Default: `JPBRIDGE.m`).
- `JP_DEBUG` – Falls auf `1` gesetzt, werden alle ausgeführten Shell-Kommandos sowie die von der Bridge ausgegebenen Marker zusätzlich auf `stderr` protokolliert.

Kommandozeilenoptionen

Neben den Umgebungsvariablen können auch Optionen direkt beim Aufruf des CLI übergeben werden. Diese überschreiben ggf. gesetzte Variablen.

- `-g/-root` – setzt den Root-Global (ohne führendes `^`, z. B. `store`).
- `-a/-r` – Debugausgabe: AST oder generierte YottaDB-Referenz anzeigen (vgl. Abschnitt 5).
- `-h/-help` – Kurzübersicht aller verfügbaren Optionen.

Zustand zurücksetzen

Ein reproduzierbarer Ausgangszustand kann in wenigen Schritten hergestellt werden:

- **Container stoppen und entfernen:**

```
docker rm -f "${YDB_CONTAINER:-ydb}" 2>/dev/null || true
```

- **Build-Artefakte neu erzeugen:**

```
./build.sh clean && ./build.sh
```

- **Container erneut starten:**

```
./run_ydb.sh
```

- **Beispieldaten neu einspielen:** Siehe Abschnitt 6.1.

Toolversionen (Referenz)

- GCC: getestet mit GCC `13.3.0`
- Flex/Bison: Flex `2.6.4`, Bison `3.8.2`
- YottaDB: Container `yottadb-debian:1.38`

Abweichungen sind grundsätzlich möglich, können aber zu abweichenden Warnungen, geänderten Linker-Flags oder leicht variierendem Laufzeitverhalten führen.

7 Evaluierung

7.1 Methodik und Metriken

Die Evaluierung verfolgt das Ziel, die Korrektheit, Robustheit und Gebrauchstauglichkeit des entwickelten Werkzeugs nachvollziehbar zu belegen. Dazu werden klare Rahmenbedingungen definiert, ein reproduzierbarer Versuchsaufbau gewählt und geeignete Metriken festgelegt.

Versuchsaufbau Die Messungen erfolgten auf einem Hostsystem mit Ubuntu 22.04. YottaDB lief isoliert in einem Docker-Container (`yottadb-debian:1.38`); das entwickelte CLI wurde aus den Quellen mit GCC 13.3.0 und Optimierungsstufe `-O2` kompiliert. Als Testdatensatz diente die in Abschnitt 6.1 angelegte Beispielstruktur. Alle Messungen erfolgten *End-to-End*, d. h. ein JSONPath-Ausdruck wurde über das CLI in einen AST übersetzt, zu einer YottaDB-Referenz transformiert und mittels `ydb -run` im Container ausgeführt.

Korrektheit Zur Überprüfung der Korrektheit wurden repräsentative JSONPath-Ausdrücke aus dem Beispieldatensatz ausgewählt. Bewertet wurde, ob diese korrekt in die erwarteten YottaDB-Referenzen abgebildet werden und den hinterlegten Wert zurückliefern.

Fehlertoleranz Für nicht unterstützte Sprachkonstrukte wurde überprüft, ob der Parser diese zuverlässig mit `EX_PARSE` abweist und dabei eine spezifische, für den Nutzer verständliche Fehlermeldung erzeugt. Damit wird sichergestellt, dass das Tool auch in Negativfällen deterministisch reagiert und keine inkonsistenten Referenzen erzeugt.

Gebrauchstauglichkeit Neben der funktionalen Korrektheit wurde die CLI auf ihre praktische Nutzbarkeit hin untersucht. Bewertet wurden die zusätzlichen Optionen (`-a` für AST-Ausgabe, `-r` für Referenzsicht, `-q` für Quiet-Mode) sowie der Interaktivbetrieb mit Prompt. Kriterium war, ob Nutzer die Zwischenergebnisse (AST, Referenz) nachvollziehbar einsehen können und Fehlermeldungen in einer für Debugging und Entwicklung verständlichen Form vorliegen.

Performanz Als Indikator für die Ausführungszeit wurde die Latenz einfacher Punktabfragen ermittelt. Dazu wurden wiederholte Aufrufe im `-q`-Modus ausgeführt, wobei die ersten Iterationen als Warmup verworfen wurden. Aus den Messreihen wurden Kennzahlen für das 50. und 95. Perzentil (*p50*/*p95*) bestimmt, um typische und seltene Verzögerungen zu quantifizieren. Dies erlaubt eine erste Einschätzung, ob das Tool im Millisekundenbereich und damit interaktiv nutzbar bleibt.

7.2 Korrektheit und Fehlertoleranz

Korrektheit - Positivfälle

Ein zentrales Kriterium für die Evaluierung ist die korrekte Übersetzung von JSONPath-Ausdrücken in die entsprechenden YottaDB-Referenzen.

Zur Demonstration wurden mehrere repräsentative Pfade aus dem in Abschnitt 6.1 beschriebenen Beispieldatensatz abgefragt. In allen Fällen erfolgte die Übersetzung korrekt, und die Abfragen lieferten den erwarteten Wert. Dabei gilt als Konvention, dass das JSONPath-Wurzelsymbol `$` standardmäßig dem Global `^store` entspricht. Beispielsweise wird der Ausdruck `$.bicycle.color` korrekt in die Referenz `^store("bicycle","color")` übersetzt.

JSONPath	YottaDB-Referenz	Erwarteter Wert
`$.bicycle.color`	`^store("bicycle","color")`	red
`$.book[0].title`	`^store("book",1,"title")`	Moby Dick
`$["book"][1]["price"]`	`^store("book",3,"price")`	22.99
`$.book[1].category`	`^store("book",2,"category")`	fiction
`$["bicycle"]["price"]`	`^store("bicycle","price")`	399

Tabelle 7.1: Korrektheit für repräsentative Pfade

Fehlertoleranz - Negativfälle

Neben den Positivfällen wurde überprüft, wie die Implementierung auf nicht unterstützte oder fehlerhafte Eingaben reagiert. Die Grundimplementierung deckt nur eine Teilmenge der JSONPath-Spezifikation ab; alle anderen Konstrukte werden deterministisch mit einem Abbruch quittiert. Der Parser liefert in diesen Fällen den Exitcode `EX_PARSE` und gibt eine spezifische Fehlermeldung aus, die die Ursache eindeutig benennt.

Die Tabelle 7.2 zeigt exemplarische Negativfälle. Damit wird sichergestellt, dass das Tool in allen nicht abgedeckten Fällen nachvollziehbar und robust reagiert.

Eingabe	**Meldung — Syntaxfehler: ...**
`$..book.title`	`Rekursiver Deszent (..) ist n.u.`
`$.book[*].title`	`Wildcard ([*]) ist n.u.`
`$.book[1:3]`	`Array-Slices ist n.u.`
`$.book[-1]`	`Index [n] erwartet (ganzzahlig, >= 0)`
`$.book[?(@.price < 10)]`	`Filter (?()) ist n.u.`
`$.book[`	`syntax error, unexpected end of file`

Tabelle 7.2: Negativfälle: tatsächliche Fehlermeldungen (erste Zeile).

7.3 Gebrauchstauglichkeit

Sichtprüfung von AST- und Referenzansicht

Ein wichtiges Kriterium der Gebrauchstauglichkeit ist die Möglichkeit, die interne Verarbeitungsschritte nachzuvollziehen. Das Tool stellt dafür verschiedene Modi bereit, welche die Transparenz der Verarbeitung erhöhen und Debugging sowie Validierung vereinfachen.

Mit gesetzter Umgebungsvariable `JP_DEBUG=1` werden zusätzlich die ausgeführten Bridge-Kommandos sowie alle empfangenen Marker auf `stderr` ausgegeben. Dadurch lässt sich die vollständige Kommunikationskette zwischen CLI, Bridge und YottaDB nachvollziehen:

```
$ export JP_DEBUG=1
$ ./jp2ydb -ar
Gib einen JSONPath-Ausdruck ein (oder 'exit'):
> $.book[0].title
> Ast-Ausgabe:
PATH
  IDENTIFIER: $
  PATH
    IDENTIFIER: book
    PATH
      LITERAL: 0
      PATH
        IDENTIFIER: title
> YottaDB-Abfrage:
[YDB] Referenz: ^store("book",1,"title")
[cmd] docker exec -i ydb bash -lc (...)
[ydb] JP:D:1
[marker] JP:D = 1
[ydb] JP:V:Moby Dick
[marker] JP:V = Moby Dick
Moby Dick
```

Ergänzend dazu existieren drei dedizierte Schalter:

- `-a` gibt den syntaktischen Abstrakten Syntaxbaum (AST) aus, wodurch die Parserergebnisse unmittelbar sichtbar werden.
- `-r` zeigt die aus dem AST abgeleitete YottaDB-Referenz.
- `-q` unterdrückt Zusatzinformationen und liefert ausschließlich den angefragten Wert. Dieser Modus ist insbesondere für automatisierte Tests oder Latenzmessungen geeignet.

Interaktive und gezielte Nutzung

Im Interaktivmodus (`-ar` ohne Argument) kann der Nutzer mehrere Pfade nacheinander eingeben und erhält jeweils AST- und Referenzausgabe. Dies erleichtert Ad-hoc-Tests und das schnelle Experimentieren mit unterschiedlichen Pfaden.

Unabhängig vom gewählten Modus wird die Anfrage stets ausgeführt, sodass am Ende zusätzlich der gefundene Wert ausgegeben wird. Die Schalter steuern lediglich, ob ergänzend der AST und/oder die YottaDB-Referenz angezeigt wird.

```
$ ./jp2ydb -a $.book[0].title   # AST + Ergebnis
PATH
  IDENTIFIER: $
  PATH
    IDENTIFIER: book
    PATH
      LITERAL: 0
      PATH
        IDENTIFIER: title
Moby Dick

$ ./jp2ydb -r '$.book[0].title' # Referenz + Ergebnis
[YDB] Referenz: ^store("book",1,"title")
Moby Dick
```

Im Quiet-Modus reduziert sich die Ausgabe auf den reinen Wert:

```
$ ./jp2ydb -q '$.book[0].title'
Moby Dick
```

Bewertung

Die Möglichkeit, zwischen detaillierten Debug-Informationen, AST-/Referenzsicht und einer minimalen Nutzwertausgabe umzuschalten, trägt wesentlich zur Gebrauchstauglichkeit bei. Nutzer können je nach Anwendungsszenario den passenden Modus wählen. Dadurch ist sowohl eine hohe Transparenz für Entwickler als auch eine schlanke Nutzerschnittstelle für den produktiven Einsatz gewährleistet.

7.4 Ergebnisse und Diskussion

Die Evaluierung betrachtet im Folgenden sowohl Leistungsaspekte (Latenz), als auch die semantische Korrektheit der Datenzugriffe ($DATA-Fälle). Damit wird überprüft, ob die in der Implementierung beschriebene Funktionsweise praktisch nachvollzogen werden kann und welche Einschränkungen sich aus der Prototyp-Natur ergeben.

Ende-zu-Ende-Latenz

Gemessen werden 10 Aufrufe eines einfachen Pfads im `-q`-Modus; die ersten 5 Aufrufe dienen als Warmup und werden verworfen.

```
q='$.store.book[0].title'
# Warmup
for i in {1..5}; do ./jp2ydb -q "$q" >/dev/null; done
# Messung
: > times.txt
for i in {1..10}; do
  /usr/bin/time -f "%e" ./jp2ydb -q "$q" >/dev/null 2>> times.txt
done
sort -n times.txt > t.sorted
n=$(wc -l < t.sorted); p50=$(( (n+1)/2 )); p95=$(( (95*n+99)/100 ))
echo "p50=$(sed -n "${p50}p" t.sorted)s  p95=$(sed -n "${p95}p"
   t.sorted)s"
```

Listing 7.1: Messskript zur Latenzbestimmung

Variante	p50 [ms]	p95 [ms]
`-q`, Pfad existent	271.5	279.7

Tabelle 7.3: End-to-End-Latenz (10 Mittelwerte à 100 Aufrufe).

Ergebnisse *Kurzinterpretation:* Schon bei nur 10 Messungen zeigt sich eine subsekündige Latenz im Bereich weniger Millisekunden.

$DATA-Semantik (d=0/1/10/11)

Diese Auswertung betrachtet nicht die Geschwindigkeit, sondern die semantische Ebene: Wie die von YottaDB bereitgestellten `$DATA`-Fälle (0/1/10/11) im Kontext von JSONPath zu interpretieren sind. *Setup:* `--root=store`. Debug-Markierungen via `JP_DEBUG=1`.

```
$ ./jp2ydb -q '$.book[2]'
[ydb] JP:D:0
[ydb] JP:V:
# d=0: Knoten nicht vorhanden (Index out of range)

$ ./jp2ydb -q '$.bicycle.color'
[ydb] JP:D:1
[ydb] JP:V:red
red
# d=1: Blatt mit Wert

$ ./jp2ydb -q '$.bicycle'
[ydb] JP:D:10
{}
# d=10: nur Kinder; Grundimplementierung gibt {} als Platzhalter
    aus
```

Listing 7.2: Nachweis der $DATA-Fälle d=0/1/10

Hinweis zu `d=11` (Wert *und* Kinder): YottaDB erlaubt an einem Knoten gleichzeitig einen Wert und untergeordnete Knoten (`$DATA=11`). *Reines JSON* kennt dieses Modell nicht: Ein Schlüssel ist entweder ein *Wert* (z. B. String/Zahl) oder ein *Objekt/Array*, aber nicht beides zugleich. Darum kann `d=11` in einem JSON-Objekt *nicht* auftreten; es ist eine Eigenschaft des YottaDB-Datenmodells.

(Optional) Technischer Nachweis von `d=11` in YottaDB Für die Vollständigkeit (außerhalb des JSON-Modells) lässt sich `d=11` so anlegen und prüfen:

```
# Wert + Kind am selben Knoten (nur YottaDB, nicht darstellbar in
    reinem JSON)
$ docker exec -i ydb bash -lc 'ydb -run %XCMD "SET
    ^store(\"bicycle\")=\"Bicycle object\", SET
    ^store(\"bicycle\",\"color\")=\"red\""'

$ ./jp2ydb -q '$.bicycle'
[ydb] JP:D:11
[ydb] JP:V:Bicycle object
Bicycle object
```

Listing 7.3: Technischer Nachweis $DATA=11 in YottaDB

In JSON müsste man das mit einem Zusatzschlüssel `"_value"` *simulieren* (kein Standard):

```
{ "store": {
    "bicycle": {
      "_value": "Bicycle object",
      "color": "red",
      "price": 399
    }
  }
}
```

Listing 7.4: Simulation eines $DATA=11-Knotens in JSON

JSONPath	JP:D	JP:V vorhanden	Ausgabe
`$.book[2]`	`0`	nein	(leer)
`$.bicycle.color`	`1`	ja	`red`
`$.bicycle`	`10`	nein	`{}`
`$.bicycle` (nur YDB, mit Root-Wert)	`11`	ja	`Bicycle object`

Tabelle 7.4: $DATA-Fälle mit vorhandenem Datensatz; `d=11` ist YottaDB-spezifisch und in reinem JSON nicht darstellbar.

Kurzüberblick (Tabelle) Die Auswertung verdeutlicht, dass die Bridge die `$DATA`-Semantik von YottaDB korrekt über Marker abbildet. Für typische JSONPath-Zugriffe sind die Fälle `d=0` (nicht vorhanden), `d=1` (Wert vorhanden) und `d=10` (nur Kinder) ausreichend, um das JSON-Datenmodell konsistent darzustellen. Der Sonderfall `d=11` tritt in JSON nicht auf und ist damit primär eine YottaDB-Spezialität, die nur durch zusätzliche Konventionen (z. B. _value-Schlüssel) abgebildet werden könnte. Damit bestätigt sich, dass die aktuelle Implementierung für JSON-kompatible Daten hinreichend ist, während diese Spezialfälle einen klaren Erweiterungspunkt für zukünftige Arbeiten darstellen.

Einschränkungen der Aussagekraft

Zum Abschluss der Evaluierung werden die Grenzen der Ergebnisse und mögliche Verzerrungen benannt, um die Aussagekraft realistisch einzuordnen.

- Messungen sind Ende-zu-Ende (CLI + Docker + `ydb -run`); keine isolierten Microbenchmarks.
- Kleiner, synthetischer Datensatz; Timing-Jitter hat relativ großen Einfluss.

- Einzelmaschine; Ergebnisse sind nicht ohne Weiteres auf eine andere Hardware/OS übertragbar.
- Keine Konkurrenzvergleiche mit alternativen JSONPath-Engines (out-of-scope).
- Die Grundimplementierung ist ein Prototyp; Stabilität, Speicherverbrauch und Skalierbarkeit wurden nicht evaluiert.
- Fokus auf lesenden Zugriff; Schreib-Operationen oder gleichzeitige Zugriffe (Concurrency) sind nicht betrachtet.
- Parser deckt nur eine Teilmenge von JSONPath ab; die Ergebnisse gelten nur innerhalb dieses Umfangs.
- Debug-/Bridge-Overhead könnte die Messwerte verfälschen (nicht vollständig kontrolliert).

8 Ausblick

Dieses Kapitel skizziert mögliche Weiterentwicklungen des Prototyps. Die dargestellten Perspektiven betreffen sowohl die funktionale Abdeckung der JSONPath-Spezifikation, die Standardkonformität und Performanz, als auch Fragen der Integration und zukünftiger Erweiterungen.

8.1 Funktionale Erweiterungen

Unterstützung weiterer JSONPath-Konstrukte. Die aktuelle Implementierung deckt nur eine Teilmenge von JSONPath ab. Zukünftig könnten auch Wildcards (`.*/[*]`), Slices `[a:b]`, rekursiver Deszent (`..`), Filterausdrücke (`?()`) sowie Unionen unterstützt werden, um die Ausdrucksmächtigkeit deutlich zu erhöhen.

Allgemeiner Evaluator. Anstelle der derzeitigen linearen Pfadtransformation wäre ein allgemeiner Evaluator denkbar, der komplexere Ausdrücke schrittweise interpretiert. Dies würde die Engine flexibler machen und eine Erweiterung auf vollständige JSONPath-Semantik ermöglichen.

Typkodierung der Subskripte. Eine optionale Typkennzeichnung der Subskripte könnte String-Keys wie `"1"` eindeutig von numerischen Indizes unterscheiden und damit die Robustheit im Umgang mit gemischten Datenmodellen verbessern.

Erweiterung der Bridge-Funktionalität. Die Bridge könnte um die Ausgabe von Kind-Schlüsseln erweitert werden (Traversal von $d = 10$-Knoten). Dadurch ließen sich vollständige JSON-Objekte rekonstruieren, anstatt nur Skalare oder leere Platzhalter auszugeben.

8.2 Standardkonformität und Performance

Annäherung an RFC 9535 (7). Ein wesentliches Ziel ist die schrittweise Orientierung an der in RFC 9535 definierten Teilmenge von JSONPath, einschließlich Abgleich mit offiziellen Test-Suiten.

Erweiterte Benchmarks. Neben Punktzugriffen sollten auch komplexere Abfragen wie Wildcard-Iterationen, Filterausdrücke und tief verschachtelte Pfade systematisch evaluiert werden.

Optimierte Datenbankzugriffe. Alternative Zugriffspfade in YottaDB, etwa Streaming-Queries oder eine optimierte Nutzung von `$QUERY`, könnten die End-to-End-Latenz weiter reduzieren.

Skalierbarkeit. Für größere Datensätze oder Multi-User-Szenarien wäre eine Analyse der Skalierbarkeit erforderlich, insbesondere im Hinblick auf Transaktionssicherheit und parallele Zugriffe.

8.3 Integration und Verbreitung

Verpackung und Distribution. Eine Veröffentlichung als Docker-Image und als Release-Binary für verschiedene Plattformen würde die Nachnutzung und Verbreitung erheblich erleichtern.

Automatisierte Tests. Die bestehende Mini-Regression-Suite könnte um automatisierte Negativfälle ergänzt und in CI-Pipelines integriert werden.

Internationalisierung. Eine optionale Übersetzung der Fehlermeldungen sowie eine konfigurierbare Ausgabetiefe für Debug-Marker würden die Anpassbarkeit an verschiedene Einsatzkontexte erhöhen.

Einbettung in Toolchains. Langfristig wäre eine Integration in bestehende Datenanalyse-Pipelines oder die Bereitstellung von Bindings für andere Programmiersprachen denkbar.

8.4 Darüber hinaus

Schreiboperationen. Eine Erweiterung über den rein lesenden Zugriff hinaus könnte optionale Schreiboperationen (`SET`/`KILL`) mit Transaktionskontrolle umfassen.

Generische Brücke für M-Datenbanken. Das Tool könnte als generische Brücke für JSONPath-ähnliche Anfragen in anderen M-basierten Datenbanken wie GT.M oder InterSystems Caché weiterentwickelt werden.

Streaming-Evaluation. Mittelfristig ist eine Streaming-Evaluation von JSONPath-Ausdrücken vorstellbar, bei der große Ergebnisse nicht vollständig im Speicher materialisiert, sondern iterativ verarbeitet werden.

Literatur

[1] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. IETF RFC 8259. Formale Spezifikation von JSON. 2017. URL: https://www.rfc-editor.org/rfc/rfc8259.

[2] GitHub Contributors. *JSONPath Plus*. Erweiterte JSONPath-Implementierung in JavaScript, kompatibel mit Node.js und Browsern. 2024. URL: https://github.com/JSONPath-Plus/JSONPath.

[3] *Docker Documentation*. Online Documentation. Containerisierung, `docker run/exec/cp`, Volumes. Docker, Inc., 2024. URL: https://docs.docker.com/.

[4] Charles Donnelly, Richard Stallman und Akim Demaille. *GNU Bison Manual*. Offizielle Bison-Dokumentation. Free Software Foundation. 2024. URL: https://www.gnu.org/software/bison/manual/.

[5] *Flex: The Fast Lexical Analyzer*. Offizielle Flex-Dokumentation. The Flex Project. 2024. URL: https://westes.github.io/flex/manual/.

[6] *GNU Make Manual*. Build-Automatisierung (Makefile). Free Software Foundation. 2024. URL: https://www.gnu.org/software/make/manual/.

[7] Stefan Gössner u. a. *JSONPath: Query Expressions for JSON*. IETF RFC 9535. Standardisierte Spezifikation von JSONPath. 2023. URL: https://www.rfc-editor.org/rfc/rfc9535.

[8] Jayway AB. *Jayway JSONPath*. Java-Bibliothek zur Auswertung von JSONPath-Ausdrücken. 2024. URL: https://github.com/json-path/JsonPath.

[9] *M Programmer's Guide*. YottaDB-Handbuch zur Programmierung in M – inklusive Globals, $DATA, $GET, Direct Mode. YottaDB, LLC. 2025. URL: https://docs.yottadb.com/ProgrammersGuide/.

[10] *PostgreSQL Documentation: JSONPath*. Referenz zur JSONPath-Unterstützung in PostgreSQL. PostgreSQL Global Development Group. 2024. URL: https://www.postgresql.org/docs/current/functions-json.html#FUNCTIONS-SQLJSON-PATH.